Swarm and Evolutionary Computation 105 (2026) 102404

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

Artificial bee colony meets variable neighborhood search: Fresh insights into influence maximization in social networks

Jianxin Tang [a,*], Fangjie Dong [a], Chen Wu [a], Wenhuan Chen [a], Huiyi Wei [a,b]

[a] School of Computer Science and Artificial Intelligence, Lanzhou University of Technology, PR China
[b] School of Computer Science and Engineering, Southeast University, PR China

ARTICLE INFO

Keywords:
Social network analysis
Influence maximization
Meta-heuristic algorithms
Artificial bee colony
Variable neighborhood search

ABSTRACT

As one of the fundamental topics within complex networks, the influence maximization problem invites the pursuit of key nodes that can maximize the influence spread. However, conventional meta-heuristics struggle to identify an optimal seed set due to rigid neighborhood generation strategies and weak directional guidance. In light of this, the artificial bee colony algorithm empowered by the variable neighborhood search strategy is deployed to address the above challenges. Specifically, an elite-global cooperative neighborhood construction mechanism that injects directional cues is devised to avoid aimless exploration. Then, a variable neighborhood search strategy that dynamically switches among a continuum of fine-grained neighborhood structures is introduced to achieve a good trade-off between exploration and exploitation. Furthermore, when the population converges prematurely to local optima, a fine-grained, individual-centric stagnation control mechanism combined with a stagnation-aware population management mechanism further maintains search vitality at both the individual and population levels. Experiments on nine real-world networks reveal that the proposed algorithm achieves an overall average improvement of 6.3% in influence spread relative to all baseline methods, demonstrating its strong scalability across different types of networks. Such findings showcase its potential for practical applications in rumor control and cascading failures.

1. Introduction

How can a single post shift public perception, yet large-scale campaigns fall silent? Such a pronounced asymmetry compels us to question how to accurately locate the source of information dissemination. It can be answered via complex network modeling that maps sophisticated social structures onto graphs. In practice, however, the effort to detect these source nodes within the modeled graph resembles traversing a maze. The challenge lies in going through the maze to find these key nodes that can trigger such widespread impact. The objective demands both advanced strategies and profound insights into network structure [1]. Such complexity not only poses computational challenges but also grants the problem remarkable value across multiple practical sectors. The ability to pinpoint influential nodes shapes public opinion into reasoned dialogue, halts the ripple of failures in power grids, tips the scales of elections, and stirs the heart of social movements [2]. In quieter corners, this subtle art empowers institutions like hospitals to weave vital health notices that shield communities and nurture well-being via social media [3].

To formalize the search mathematically, scholars in the domain of complex networks framed the issue as the influence maximization (IM) problem. The goal is to select initial users within the network, often termed seeds, that maximize the influence spread under specific propagation models [4]. However, it is far from trivial to identify the optimal seed set. Such difficulty arises because every node exhibits a distinct influence spreading capability that determines its potential to affect neighbors. The hurdle lies not just in finding out influential nodes, but in determining how their individual potentials combine to maximize the overall spread.

The IM problem was first defined as a discrete combinatorial optimization problem with its NP-hardness proved by Kempe et al. [5]. It indicates that finding an optimal solution within polynomial time is intractable. Early efforts focused on greedy algorithms with provable approximation guarantees, but such algorithms are burdened by the weight of extensive Monte Carlo simulations to select influential nodes one by one. In pursuit of greater computational efficiency, heuristic methods [6] that draw upon network topology features have been developed to avoid costly simulations. However, the reliance on direct top-ranked selection inherently overlooks influence overlap within

* Correspondence to: No. 287 Langongping Road Qilihe District, Lanzhou City, Gansu, PR China.
E-mail addresses: tangjx@lut.edu.cn (J. Tang), fangjie@lut.edu.cn (F. Dong), wuchen27@lut.edu.cn (C. Wu), whchen@lut.edu.cn (W. Chen), weihy@lut.edu.cn (H. Wei).

https://doi.org/10.1016/j.swevo.2026.102404
Received 23 January 2026; Received in revised form 29 March 2026; Accepted 26 April 2026
Available online 5 May 2026
2210-6502/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

seed sets, thereby undermining performance. To bridge this gap, deep learning based methods [7] explicitly integrate the graph topology to mitigate such influence overlap. Nevertheless, the gain in precision comes with the drawback of prohibitively high training costs.

Distinct from these approaches, meta-heuristics that operate independently of problem characteristics have long been valued for tackling NP-hard problems [8]. Such algorithms mimic the subtle ingenuity of nature to traverse solution spaces with a dynamic equilibrium between exploration and exploitation. Many meta-heuristics have been successfully applied to IM with promising results, including natural evolution strategies such as the memetic algorithm [9], swarm intelligence behaviors like harris' hawks optimization [10], and physical processes exemplified by the gravitational search algorithm [11]. Despite their prevalence, such algorithms tend to yield seed sets with low accuracy due to three primary flaws. Firstly, blind exploitation arises from the absence of directional guidance. Secondly, single neighborhood search strategies erode population diversity. Thirdly, curtailed exploration in later iterations amplifies the risk of premature convergence.

To address the aforementioned challenges, a collaborative optimization is pioneered, termed the artificial bee colony algorithm empowered by the variable neighborhood search strategy (ABCVNS). The artificial bee colony algorithm (ABC) [12] is adopted as the base optimizer due to the alignment between its perturbation logic and the node replacement operation in IM. However, ABC's inherent randomness often traps the search in directional blindness. To rectify such low-efficiency disturbances, an elite-global cooperative neighborhood construction (EGCNC) mechanism serves as a directional engine that applies elite knowledge to fine-tune search direction recursively. Although the EGCNC mechanism enhances exploitation, effective optimization requires a good trade-off between exploitation and exploration. Thus, the variable neighborhood search (VNS) strategy [13] is introduced to achieve such a trade-off. The VNS strategy enables dynamic switching among neighborhood structures, unlike existing meta-heuristics that are confined to a single and fixed neighborhood search pattern. Such flexibility in the neighborhood structures augments ABCVNS's capacity to escape local optima.

Moreover, a dual-level scheme is adopted to mitigate the risk of premature convergence. At the individual level, a fine-grained, individual-centric stagnation control mechanism is devised by ABC's unique abandonment strategy that resets a specific failing individual without disrupting the population in real-time. At the population level, a stagnation-aware population management mechanism responds to the diversity decay. Finally, a robust, interpretable, and generalizable solving paradigm is established to solve the IM and open a universal pathway to tackle other combinatorial optimization problems. The major contributions of our method are described as follows:

- The ABC empowered by VNS is introduced as a novel optimization to navigate the rugged IM landscape. Specifically, diverse neighborhood structures are dynamically switched to overcome the local optima traps and underpin a robust balance between exploitation and exploration.
- The directional blindness of standard ABC is resolved by an elite-global cooperative neighborhood construction (EGCNC) mechanism to transform the search from aimless wandering into elite-oriented fine-tuning.
- A fine-grained, individual-centric stagnation control mechanism is established by uniquely employing ABC's abandonment strategy to bridge the gap in resolving the individual stagnation in real-time.

The remainder of the paper is organized as follows. To start with, Section 2 reviews relevant literature. Section 3 details the proposed ABCVNS. Section 4 reports the experimental results and analysis. Section 5 discusses strengths, weaknesses, and potential applications, followed by a brief conclusion in Section 6.

2. Related work

2.1. Approximate methods

Kempe et al. [5] set forth a hill-climbing greedy algorithm for the IM, crowned with a $(1-1/e-\epsilon)$ approximation guarantee, where ϵ accounts for the error introduced by Monte Carlo simulations. However, the need for such extensive simulations per node to compute marginal gains incurred $O(kRmn)$ complexity, where k, R, n, and m denote the seed set size, the simulation count, the number of nodes, and edges in the graph, respectively. Leskovec et al. [14] answered the time hurdle with the Cost-Effective Lazy-Forward algorithm (CELF). The authors exploited the submodularity of the objective function to prune redundant computations and accomplished a 700-fold speedup. But repeated marginal gain computations persisted as a scalability defect. Chen et al. [15] extended the method by a node exclusion rule to further lower complexity to $O(kRm)$ under the well-established independent cascade (IC) model. Even with such enhancements, simulation-based methods remain computationally expensive for large-scale networks.

A paradigm shift occurred with Borgs et al. [16], who resolved the barrier with reverse influence sampling (RIS) and marked the first near-linear-time algorithm with theoretical guarantees. Since then, subsequent research has focused on optimizing RIS efficiency. Notably, Tang et al. [17] implemented a two-phase influence maximization method to reduce time complexity, later devising the improved version IMM based on martingale estimation technology [18]. And Guo et al. [19] optimized reverse reachable set generation via subset sampling and sentinel strategies to drastically reduce both the sampling overhead and the average size of reverse reachable sets. Despite their accuracy, RIS-based methods exhibit scalability constraints dominated by memory space. This is because all generated reverse reachable sets must be retained in memory for the subsequent seed selection phase.

At the core, the first category of paradigms remains undermined by dual implementation barriers: prohibitive computational expenses from repeated simulations and sustainable memory demands. These shortcomings highlight the pressing need for algorithms that balance accuracy and scalability.

2.2. Heuristic methods

Heuristic methods respond to the efficiency bottlenecks by screening influential nodes based on topological metrics ranging from local to semi-local and global. Local metrics, e.g., degree centrality [20], are simple and fast but fail to capture influence beyond immediate neighbors. Conversely, global indicators like closeness centrality [21] assess node influence more precisely, albeit with prohibitive computational complexity. To this end, Chen and Su [22] engineered a novel metric named DC+ and the corresponding gravity model extensions DCGM+, which quantified influence via the node and neighbor degrees to resolve ambiguities in uniform-degree nodes. Recently, Zhu and Hao [23] fused intrinsic centrality combining k-shell with degree information, with extrinsic centrality capturing neighbor contributions. It achieved linear complexity $O(m)$, though its performance remained partially constrained by the sensitivity of k-shell decomposition. While automatic heuristic design has been applied to many NP-hard problems [24], research in IM has primarily focused on manually engineering composite metrics.

Heuristic methods offer a tractable path toward the IM in large-scale networks. However, a critical limitation is that heuristics directly select the top-k highest-scoring nodes as the seed set without adequately considering potential influence overlap among candidates. The oversight constrains propagation effectiveness and makes performance sensitive to network topology.

2.3. Meta-heuristic methods

Characterized by low dependency on prior knowledge and sweeping global search capabilities, meta-heuristics [25] have emerged to overcome structural sensitivity and influence overlap in heuristics. Based on their underlying principles, meta-heuristics can be divided into three classes: physics-based algorithms, evolutionary algorithms, and swarm intelligence algorithms.

Physics-based algorithms constitute the first primary class, with simulated annealing (SA) being the most representative example. Jiang et al. [26] pioneered its application to IM and proposed the expected diffusion value (EDV) function to bypass costly simulations. Although SA theoretically balances exploration and exploitation via temperature control, its fixed cooling rate and sensitivity to the initial temperature compromise robustness. Later variants incorporating adaptive temperature tuning [27] were designed to optimize both convergence rates and stability.

Inspired by Darwin's theory of evolution, evolutionary algorithms emulate genetic operators to guide the search process. Qiu et al. [28] devised LIDDE, a local influence decreasing differential evolution algorithm that considers fourth-order neighbor evaluation to sharpen precision. But the fixed candidate pool sizes constrain adaptability. Zhu et al. [29] addressed the flaws with a phased evaluation-enhanced approach, PHEE, by constructing a dynamic pool sizing based on network scale. Parameter sensitivity, however, remains a drawback. More recently, Tang et al. [30] augmented the differential evolution algorithm with fitness landscape analysis and directed the search via entropy, albeit with added landscape sampling costs.

Distinct from evolutionary algorithms, swarm intelligence algorithms exploit swarm behaviors rather than genetic mutation to navigate the solution space. Inspired by the foraging behaviors of birds, Gong et al. [31] presented a discrete particle swarm optimization algorithm and constructed a two-hop influence evaluation function to estimate the influence spread of candidate seeds, though computationally expensive. To expedite convergence, He et al. [32] proposed a multisearch particle swarm optimization algorithm that incorporates learning automata to coordinate multiple search strategies. Beyond particle swarm optimization, ABC has emerged as another prominent swarm intelligence algorithm. Improvements to ABC primarily target evaluation function design and search strategy optimization. As for assessment accuracy, Ma and Liu [33] developed an adaptive discrete ABC algorithm (ADABC) that incorporates a three-layer evaluation strategy, but its computational overhead hinders scalability. Another improvement by Sheikhahmadi and Zareie [34] architected a multi-objective ABC (MOABC) that embedded a dynamic cross-variation mechanism to bolster search capabilities.

In summary, despite the progress achieved, existing approaches still face the following shortcomings. First, blind exploitation arises from the absence of directional guidance. Second, single neighborhood search strategies erode population diversity. Third, curtailed exploration in later iterations amplifies the risk of premature convergence. All these shortcomings limit the upper bound of the algorithm's solution accuracy.

3. The proposed algorithm: ABCVNS

3.1. Preliminaries

A social network can be modeled as a graph $G = (V, E)$, where the set of vertices V denotes the individuals and the edge set E represents their mutual relationships. The IM problem aims to select a subset of nodes within a given network, referred to as a seed set S, to trigger the widest possible spread under specific propagation models, formalized as Eq. (1).

$$S = \underset{S\subseteq V, |S|\le k}{\arg\max}\ \sigma(S) \tag{1}$$

where $\sigma(S)$ denotes the influence spread of the seed set S.

In our paper, the IC model [5] is employed to simulate the information diffusion process. Nodes within the IC model are classified as either active or inactive. An active node independently attempts to activate its inactive neighbors with a given probability, with each attempt occurring only once. If successful, the neighbor attempts to activate its inactive neighbors in the next round. The process repeats until no further activations occur.

The local influence estimation (LIE) function [31] serves as the fitness function to approximate the influence spread of a candidate seed set, thereby eliminating the need for costly repeated Monte Carlo simulations. LIE measures the expected number of nodes activated within the two-hop neighborhood of the seed set, formulated in Eq. (2), to supersede EDV, which only considers one-hop neighbors for precise influence estimation. The final solution is obtained by optimizing LIE with the proposed search strategy.

$$\begin{aligned} LIE(S) &= \sigma_0(S) + \sigma_1^*(S) + \widetilde{\sigma}_2(S) = k + \sigma_1^*(S) + \frac{\sigma_1^*(S)}{|N_1(S)\setminus S|}\sum_{u\in N_2(S)\setminus S} pd_u^* \\ &= k + \left(1 + \frac{1}{|N_1(S)\setminus S|}\sum_{u\in N_2(S)\setminus S} pd_u^*\right)\sigma_1^*(S) \\ &= k + \left(1 + \frac{1}{|N_1(S)\setminus S|}\sum_{u\in N_2(S)\setminus S} pd_u^*\right)\sum_{i\in N_1(S)\setminus S}\left(1 - \prod_{(i,j)\in E, j\in S}(1-p_{ij})\right) \end{aligned} \tag{2}$$

Here, $N_1(S)$ and $N_2(S)$ denote the one-hop and two-hop neighbor sets of S, respectively. The probability of node i activating node j, p_{ij}, is set to a constant p. d_u^* counts edges connecting node u to nodes in $N_1(S)$ and $N_2(S)$ and serves to estimate the activation probability of u.

3.2. Framework overview

A schematic overview is portrayed in Fig. 1. The algorithm preserves the ABC cycle and applies the VNS strategy to the global best solution G_{best} to identify the optimal seed set via the following six stages.

(1) The workflow begins with (a) Population Initialization (See Section 3.3.1), where candidate seeds are derived by a fusion of degree-based heuristics and random sampling.
(2) Employed bees in (b) are updated via the proposed elite-global cooperative neighborhood construction (EGCNC) mechanism, as formulated in Eqs. (5)–(7). This mechanism adaptively integrates the current solution with G_{best} or elites if available. Section 3.3.2 details the employed bees phase, with the corresponding pseudocode provided in Alg. 1.
(3) Onlooker Bees in (c) perform tournament selection and refine candidates via the EGCNC mechanism (see Eqs. (5)–(7)) to form new solutions, as described in Section 3.3.3 and Alg. 2.
(4) Scout Bees in module (d), elaborated in Section 3.3.4, promote exploration by replacing stagnated solutions with a probability of 0.5 using random or elite-guided strategies, with the full execution flow provided in Alg. 3.
(5) The VNS strategy refines $G_{\textbf{best}}$ via shaking (e) and local search (f). For further details, refer to Section 3.4 and Alg. 4. In module(e), red, blue, and green lines denote solution improvement, no improvement, and unconditional progression, respectively.
(6) To maintain search vitality, population management (g) is triggered when progress stalls, outlined in Alg. 5. The underlying logic of the proposed stagnation-aware population management mechanism is detailed in Section 3.5.

3.3. Artificial bee colony algorithm

3.3.1. Population initialization

The original ABC [12] initializes food sources $\{x_i \mid i = 1, 2, \ldots, SN\}$ by uniform sampling, where jth dimension of solution x_i is defined as:

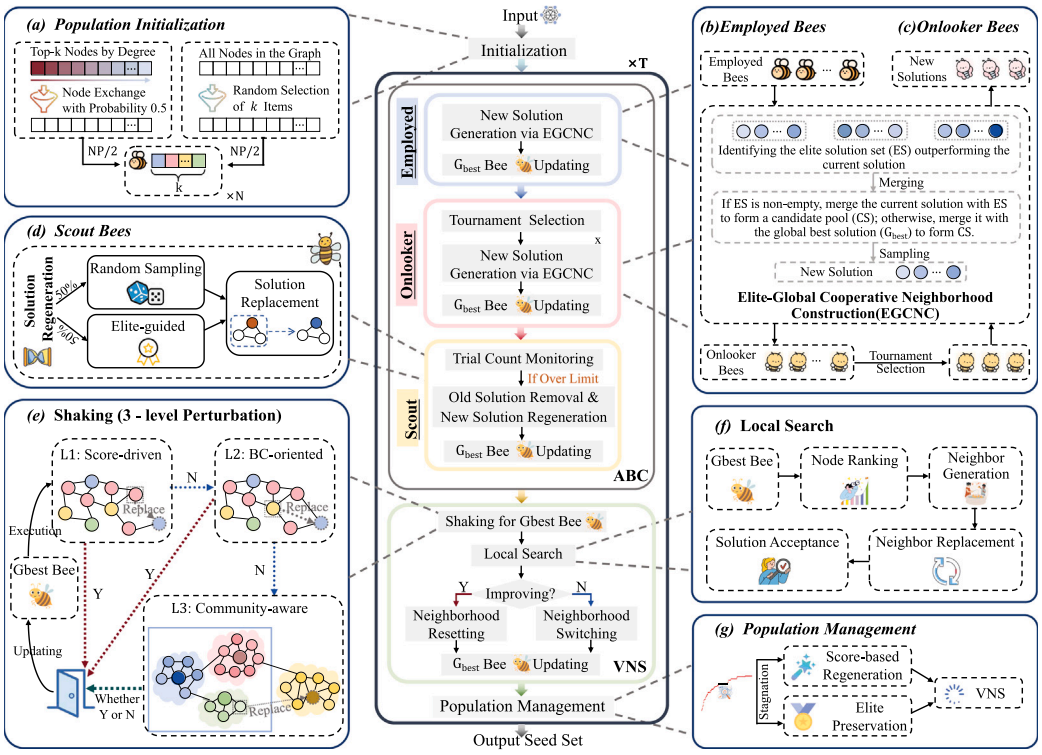


Fig. 1. The framework of ABCVNS. Note that the same bee icon is used to represent a particular bee type.

$$x_{ij} = LB_j + rand(0,1) \cdot (UB_j - LB_j) \tag{3}$$

where LB_j and UB_j denote the lower and upper bounds of the jth dimension, and $rand(0,1)$ is a uniformly distributed random variable over $[0,1]$.

Adapted for the IM, each food source corresponds to a candidate solution $x_i = \{v_{i,1}, v_{i,2}, \ldots, v_{i,k}\} \subseteq V$, formulated via a set-based encoding strategy of k distinct nodes. A naive random selection of k nodes as per Eq. (3) tends to produce inferior solutions and premature convergence. Hence, the initialization phase adopts a well-designed initialization strategy that partitions the population into two groups of equal size to establish a well-distributed starting point for the ensuing global search. The first half leverages degree centrality by selecting the top-k nodes, where each is replaced with a 0.5 probability by another node from the remaining to embed high-quality solutions and infuse stochastic variability. Meanwhile, to curb the bias toward high-degree nodes, a uniform sampling from V is used to create the second subset. Subsequently, the fitness of these solutions is assessed as per Eq. (2).

3.3.2. Employed bees

In the standard ABC [12], each employed bee explores the neighborhood of its food source x_i by updating a random dimension j to generate a candidate v_i, as defined in Eq. (4):

$$v_{ij} = x_{ij} + \varphi_{ij}(x_{ij} - x_{kj}) \tag{4}$$

where $k \neq i$ denotes a randomly selected solution index, and $\varphi_{ij} \sim U[-1,1]$ controls the magnitude of variation.

While the aforementioned vector-based perturbation works well for continuous optimization problems, it is not directly applicable to combinatorial optimization problems like IM, where solutions are represented as sets of nodes. More importantly, the neighborhood generation strategy in standard ABC (Eq. (4)) remains driven by the stochastic difference between two random individuals. The absence of elite guidance in such a reliance on arbitrary differences impedes the discovery of promising areas and is prone to premature convergence. Existing improvements, such as the comprehensive-learning guided updating rule in ADABC [33], attempt to bridge this gap but are hindered by their sensitivity to the population sampling ratio. In particular, the use of a fixed sampling proportion across different networks limits the algorithm's performance ceiling.

To overcome these drawbacks, the EGCNC mechanism is devised to generate candidates. Unlike traditional methods that rely on blind perturbations, the EGCNC mechanism explicitly integrates the current solution with informative structures from better-performing individuals. The integration is realized through a three-stage process. First, an elite set ES is dynamically constructed to identify superior individuals. Second, a candidate pool CS is adaptively established through a set-based union of the current solution and elite structures. Finally, a new candidate is derived via uniform sampling from CS, ensuring a functional fusion of existing knowledge and elite guidance. The implementation details for each stage are as follows.

Firstly, let ES denote the set of elite solutions in the population P whose influence, measured by the LIE function (Eq. (2)), exceeds that of the current solution S_i, as formalized in Eq. (5):

$$ES = \{S_j \in P \mid LIE(S_j) > LIE(S_i)\} \tag{5}$$

Here, $S_j \in P$ denotes an arbitrary solution in the population.

Secondly, depending on whether ES is empty, a candidate pool CS is then adaptively constructed to either exploit elite information in ES or follow the global best direction. Specifically, the construction process is divided into the following two cases.

Case 1 ($ES \neq \emptyset$). When ES is non-empty, the candidate pool CS is obtained through the set union of S_i and all superior solutions in ES, as defined in Eq. (6). Firstly, the node sets of all elite solutions in ES are aggregated, and then merged with the node set of the current solution S_i to construct CS. The aggregation effectively concentrates the search space on a subset of high-potential nodes that have already demonstrated superior influence in the current population. By merging these

elite structures, the mechanism facilitates the use of high-quality solutions, significantly increasing the probability of generating a candidate that outperforms S_i.

Case 2 ($ES = \emptyset$). When ES is empty, it implies that the current solution S_i is the best-performing individual within the current population. In such a scenario, the search for S_i loses the guidance provided by superior peers, degenerating into an aimless random walk, and is prone to stagnation. To re-establish a directional pressure, the candidate pool CS is built by the union of S_i and the global best solution G_{best}, as shown in Eq. (6). Such a strategy ensures that the search remains directed toward the most promising region discovered so far, preventing the algorithm from wandering aimlessly in low-quality regions.

$$CS = \begin{cases} S_i \cup \left(\bigcup_{S_j \in ES} S_j \right), & \text{if } ES \neq \emptyset \\ S_i \cup G_{\text{best}}, & \text{otherwise} \end{cases} \tag{6}$$

Thirdly, once CS is established, a new candidate S_i' is derived by uniformly sampling k distinct nodes from the pool CS, as expressed in Eq. (7):

$$S_i' = \text{RandomSample}(CS, k) \tag{7}$$

The sampling procedure allows the new solution to inherit a portion of nodes from the current solution, while simultaneously absorbing beneficial components from superior individuals.

The effectiveness of the EGCNC mechanism is rooted in three key principles. First, search efficiency is enhanced through search space pruning. The EGCNC mechanism prunes the search space by restricting candidates to a high-potential pool CS composed of nodes from elite solutions. The search ignores less promising nodes, thereby accelerating convergence toward high-quality regions. Second, algorithm performance is improved through the recombination of advantageous genes. Since the candidate pool CS aggregates nodes proven effective in superior solutions, the sampling process from CS acts as a recombination of advantageous genes. Instead of trying a random new node, the algorithm combines successful genes from multiple superior peers. Such collective guidance increases the probability of discovering superior solutions, leading to a higher final influence spread. Third, search capability is strengthened via adaptive guidance. The adaptive switching between ES and G_{best} ensures consistent search pressure to keep the search directed toward the promising region discovered so far. Simultaneously, the stochastic nature of the sampling from CS introduces a necessary variation to escape local optima and maintain population diversity. The efficacy of the EGCNC mechanism is empirically validated in Section 4.3.The integration of EGCNC into the employed bee phase is detailed in Alg. 1. The employed bee in ABCVNS generates a new candidate solution S_i' via Eqs. (5)–(7). If S_i' exhibits improved fitness, it replaces the current solution and triggers a reset of the trial counter TM_i. If not, TM_i increments. Whenever a newly generated candidate exceeds G_{best}, the latter is updated.

3.3.3. Onlooker bees

Onlooker bees in the original ABC [12] utilize the communicated knowledge from the employed bees to select superior food sources via roulette wheel selection (Eq. (8)) and conduct further search.

$$p_i = \frac{fit_i}{\sum_{k=1}^{SN} fit_k} \tag{8}$$

where $fit(\cdot)$ denotes the fitness function. From this equation, it is evident that such a preference intensifies the exploitation of top-performing individuals but makes the swarm prone to being confined in local basins.

Distinct from roulette wheel selection, the present work adopts a tournament selection mechanism to maintain moderate pressure and wider sampling by imposing a softer bias toward top performers. As shown in Alg. 2, each onlooker bee identifies the best-performing candidate as the base solution S_{target} from a random ts-sized subset $\mathcal{T} \subseteq P$ and generates a new solution S' via Eqs. (5)–(7). A superior candidate replaces S_{target} and resets the corresponding TM_{target}, while an inferior one increments it. Whenever a new peak emerges, G_{best} is refreshed to guide the swarm to adjust its trajectory.

Alg. 1: Employed bees(G, k, P, G_{best}, TM)

Input: Graph $G = (V, E)$; Seed set size k; Population P of size NP; The global best solution G_{best}; Trial counters TM.
Output: Population P, global best G_{best} and trial counters TM.

1 **for** $i \leftarrow 1$ **to** NP **do**
2 Generate S_i' relative to S_i using Eqs. (5)–(7) ;
3 **if** $LIE(S_i') > LIE(S_i)$ **then**
4 $S_i \leftarrow S_i'$;
5 $TM_i \leftarrow 0$;
6 **if** $LIE(S_i') > LIE(G_{best})$ **then**
7 $G_{\text{best}} \leftarrow S_i'$
8 **else**
9 $TM_i \leftarrow TM_i + 1$
10 **return** P, G_{best}, TM

Alg. 2: Onlooker bees(G, k, P, G_{best}, ts, TM)

Input: Graph $G = (V, E)$; Seed set size k; Population P of size NP; The global best solution G_{best}; Tournament size ts; Trial counters TM.
Output: Updated population P; Updated global best G_{best}; Updated trial counters TM.

1 **for** $i \leftarrow 1$ **to** NP **do**
  `// Tournament selection`
2 Randomly select ts individuals from P to form tournament set $\mathcal{T}$;
3 $S_{target} \leftarrow$ The element in $\mathcal{T}$ that maximizes $LIE(S)$;
4 Generate a new solution S' relative to S_{target} using Eqs. (5)–(7) ;
5 **if** $LIE(S') > LIE(S_{target})$ **then**
6 $S_{target} \leftarrow S'$;
7 $TM_{target} \leftarrow 0$;
8 **if** $LIE(S') > LIE(G_{best})$ **then**
9 $G_{\text{best}} \leftarrow S'$
10 **else**
11 $TM_{target} \leftarrow TM_{target} + 1$
12 **return** P, G_{best}, TM

3.3.4. Scout bees

The traditional ABC [12] discards a solution S_i if it fails to improve within L trials. The corresponding bee then transforms into a scout bee and replaces S_i with a new solution S_i' acquired by pure random sampling, as per Eq. (4). The approach promotes exploration but tends to wander blindly, provides no quality assurance, and often meanders through inefficient search paths.

In contrast, the scout bees in ABCVNS engage in a bifurcated strategy to imbue search with direction, as detailed in Alg. 3. If the generated random number $r \in (0, 1) < 0.5$, a portion of the new candidate inherits nodes from the intersection of the current solution S_i and the global best G_{best} (line 5), denoted as $\mathcal{MS} = S_i \cap G_{\text{best}}$, while the remainder is drawn randomly from the complementary set $V \setminus \mathcal{MS}$ (line 6). Otherwise, a new candidate S_i' is born purely from chance (line 8) to infuse the swarm with renewed diversity. Formally, the process is

expressed as Eq. (9):

$$S_i' = \begin{cases} \mathcal{MS} \cup \text{RandomSample}(V \setminus \mathcal{MS}, k - |\mathcal{MS}|), & \text{if } r < 0.5, \\ \text{RandomSample}(V, k), & \text{otherwise.} \end{cases} \tag{9}$$

where V denotes the node set, and k is the seed set size.

Upon creation, S_i' unconditionally replaces S_i and resets the corresponding trial counter TM_i. Additionally, G_{best} is updated whenever S_i' excels it. Such a bifurcated strategy transforms a scout bee from a wandering traveler into an informed navigator, equipping the algorithm with an ability to transcend local optima through a balanced trade-off between exploitation and exploration. Finally, an individual-centric stagnation control mechanism is realized through the proposed bifurcated scout bee strategy.

Alg. 3: Scout bees(G, k, P, G_{best}, L, TM)

Input: Graph $G = (V, E)$; Seed set size k; Population P of size NP; The global best solution G_{best}; The trial limit L; Trial counters TM.

Output: Population P, global best G_{best} and trial counters TM.

```
for i ← 1 to NP do
    if TM_i ≥ L then
        Generate a random number r ∈ [0, 1] ; // see Eq. (9)
        if r < 0.5 then
            MS ← S_i ∩ G_best ;
            RS ← RandomSample(V \ MS, k − |MS|) ;
            S_i' ← MS ∪ RS ;   // Elite-guided strategy
        else
            S_i' ← RandomSample(V, k) ;   // Random strategy
        S_i ← S_i' ;
        TM_i ← 0 ;
        if LIE(S_i') > LIE(G_best) then
            G_best ← S_i'
return P, G_best, TM
```

3.4. Variable neighborhood search

The fixed, single neighborhood search pattern used in the employed and onlooker bee phases of the fundamental ABC restricts effectiveness in sophisticated landscapes. Prompted by the insight, a VNS-oriented neighborhood search strategy is embedded into the ABC to help the algorithm search the space efficiently and improve solution quality.

3.4.1. Principles

The core principle of the VNS strategy [13] is to systematically change neighborhood structures during the search process. It is grounded on three key facts [35]. First, a local optimum with respect to one neighborhood structure may not be a local optimum for another. Second, a global optimum is a local optimum with respect to all possible neighborhood structures. Third, for many problems, local optima with respect to one or several neighborhoods are relatively close to each other. The VNS strategy leverages these facts by alternating among neighborhood structures of varying scales, thereby possessing significant potential to escape local optima and explore broader regions of the solution space. Such an ability is particularly vital for the IM problem, which is characterized by multi-modality and rugged landscapes. In such a complex environment, a single-scale neighborhood search is highly susceptible to premature convergence. By contrast, the multi-neighborhood search of VNS provides a robust mechanism to facilitate traversal of the rugged terrain, reducing the likelihood of being trapped in stagnation.

Generally, VNS variants are distinguished by their neighborhood switching strategies and solution acceptance rules. Common neighborhood switching strategies include *sequential change step* and *cyclic change step* [36]. In the progressive stage, the algorithm adopts the *sequential change step* to facilitate a stable, progressive search. However, in the plateau stage, the switching shifts to a *cyclic change step* to explore diverse neighborhoods, entailing extra time, as detailed in Section 3.5. To mitigate such an overhead and ensure efficiency, the study uses the *first improvement strategy* rather than the computationally intensive *best improvement strategy* [37].

3.4.2. Neighborhood structure design

A three-layer neighborhood hierarchy ($\mathcal{L}_1$, $\mathcal{L}_2$, $\mathcal{L}_3$) is adopted to balance exploration capacity with computational overhead. On one hand, a structure with too few layers narrows the search spectrum and weakens the capacity to break free from local traps, potentially compromising solution quality. On the other hand, an excessive number of layers incurs redundant switching overhead due to frequent structure switching, diminishing the algorithm's efficiency. Therefore, the proposed three-layer hierarchy embodies a deliberate trade-off between perturbation granularity and computational tractability.

Each layer is theoretically aligned to a specific structural scale of complex networks. Such a hierarchical alignment between search operators and network topology enables the search to transcend single-scale limitations. While all three layers employ a node-replacement mechanism, they differ fundamentally in the topological scope of the candidate pool and the evaluation metrics. Specifically, $\mathcal{L}_1$ focuses on a fine-grained refinement within the first-hop neighborhood. $\mathcal{L}_2$ broadens the horizon to the two-hop neighborhood, targeting bridges invisible to $\mathcal{L}_1$. Unlike $\mathcal{L}_1$ and $\mathcal{L}_2$, which are constrained by proximity to S, $\mathcal{L}_3$ enacts jumps across communities to push the search into distant regions. The transition from $N_1(S)$ to $N_2(S)$ and further to cross-community candidate sets introduces progressively larger transformation radii to systematically expand the reachable solution regions. Such a multi-scale transition mechanism allows the search to broaden the exploration horizon, encompassing solutions that are inaccessible to single-neighborhood searches, thereby reducing the likelihood of premature convergence. The detailed operational mechanisms and search operators for each layer are elaborated below, beginning with the fine-grained refinement of $\mathcal{L}_1$.

Level-1 neighborhood $\mathcal{L}_1$: Score-driven. $\mathcal{L}_1$ acts as a fine-grained refinement operator. It promotes incremental improvement without significantly disrupting the current solution structure by exchanging a single node with a more promising alternative. The replacement relies on a composite scoring function $Score$ that integrates two distinct, complementary metrics, i.e., Degree and PageRank. The rationale for selecting these specific metrics lies in their complementary roles in modeling influence propagation dynamics. Degree focuses on the quantity of connections, quantifying the propagation potential of a node within its immediate neighborhood. Conversely, PageRank captures the propagation potential by evaluating a node's importance based on the quality of its neighbors. $Score$ integrates the quantity and quality of neighbors to provide a comprehensive assessment of node importance. The $Score$ of v is given in Eq. (10):

$$Score(v) = \alpha \cdot \text{Degree}(v) + (1 - \alpha) \cdot \text{PageRank}(v) \tag{10}$$

where α is a tunable coefficient fixed at 0.5. Here, Degree and PageRank are both normalized metrics to the range $[0, 1]$.

The implementation steps of $\mathcal{L}_1$ are as follows.

(1) **Score calculation.** The $Score$ is evaluated for all nodes within the current solution S and its direct neighbors according to Eq. (10).
(2) **Removal node identification.** The node $v_l \in S$ characterized by the minimum $Score$ value is identified for removal, as defined in Eq. (11).

$$v_l \in S, \quad \text{s.t.} \quad Score(v_l) = \min_{v \in S} Score(v) \tag{11}$$

(3) **Insertion node identification.** The search for an insertion candidate is conducted within the first-order neighborhood $N_1(S) \setminus S$, where the node v_h with the maximum $Score$ value is prioritized as the most promising replacement, as given in Eq. (12).

$$v_h \in N_1(S) \setminus S, \quad \text{s.t.} \quad Score(v_h) = \max_{v \in N_1(S) \setminus S} Score(v) \tag{12}$$

(4) **Solution update.** The refined solution S' is generated via a nodal swap operation, replacing v_l with v_h as formalized in Eq. (13).

$$S' = (S \setminus \{v_l\}) \cup \{v_h\} \tag{13}$$

Level-2 neighborhood $\mathcal{L}_2$: BC-oriented. $\mathcal{L}_2$ performs aggressive exploration of the network by integrating second-order neighborhoods $N_2(S)$ with a selection mechanism driven by betweenness centrality (BC). The transition to a two-hop search radius provides the necessary topological reach to overcome the structural confinement of $\mathcal{L}_1$. Furthermore, the selection metric shifts from $Score$ to BC, which acts as a precise filter to pinpoint pivotal bridge nodes connecting disparate network components. In contrast to the guided removal in $\mathcal{L}_1$, $\mathcal{L}_2$ employs a stochastic removal strategy to prevent the search from being trapped in local optima due to over-exploitation of the $Score$ metric. $\mathcal{L}_2$ is intended to transcend structural bottlenecks and access remote regions that remain inaccessible to one-hop neighborhoods, serving as a strategic complement to $\mathcal{L}_1$. The implementation steps of $\mathcal{L}_2$ are as follows.

(1) **Search space definition.** The search space is expanded to the second-order neighborhood $N_2(S)$, which includes all nodes reachable within two hops from the current solution S.
(2) **Removal node identification.** A node v_r from the current solution S is stochastically selected for removal.
(3) **Insertion node identification.** The candidate pool is designated as $N_2(S) \setminus S$, from which the node v_b with the highest BC value is identified as the most suitable candidate for insertion, as given in Eq. (14).

$$v_b \in N_2(S) \setminus S, \quad \text{s.t.} \quad \mathrm{BC}(v_b) = \max_{u \in N_2(S) \setminus S} \mathrm{BC}(u) \tag{14}$$

where $\mathrm{BC}(v)$ is the betweenness centrality of node v.
(4) **Solution update.** The refined solution S' is generated via a nodal swap operation, substituting v_r with v_b as formalized in Eq. (15).

$$S' = (S \setminus \{v_r\}) \cup \{v_b\} \tag{15}$$

Level-3 neighborhood $\mathcal{L}_3$: Community-aware. $\mathcal{L}_3$ enforces bold transitions by weaving structural diversity across communities into the evolving solution to uncover globally competitive solutions. Unlike $\mathcal{L}_1$ and $\mathcal{L}_2$, which are constrained by proximity to S, $\mathcal{L}_3$ selects candidates from distant modules based on the sum of the PageRank values in the community. Such a community-aware replacement facilitates exploration across community boundaries to discover high-quality solutions. The implementation steps of $\mathcal{L}_3$ are as follows.

(1) **Search space partitioning.** Identify all communities $\mathcal{C}$ in the network and determine the covered community set $\mathcal{I}(S)$ (containing nodes in S). Simultaneously, nodes within S are prioritized based on their BC values, and the top-r% nodes are identified to constitute a set $\mathcal{H}$, with r defined as the retention ratio.
(2) **Removal node identification.** From what remains $S \setminus \mathcal{H}$, the node v_s with the lowest $Score$ is selected for eviction, as defined in Eq. (16).

$$v_s \in S \setminus \mathcal{H}, \quad \text{s.t.} \quad Score(v_s) = \min_{v \in S \setminus \mathcal{H}} Score(v) \tag{16}$$

(3) **Insertion node identification.** An uncovered community $C^* \in \mathcal{C}$ is selected based on its maximum aggregate PageRank, from which the node $v_p \in C^*$ with the highest PageRank is designated as the optimal candidate for insertion, as given in Eq. (17)

$$\begin{cases} C^* \in \mathcal{C} \setminus \mathcal{I}(S), \quad \text{s.t.} \quad \sum_{u \in C}^{*} \mathrm{PageRank}(u) = \max_{C_i \in \mathcal{C} \setminus \mathcal{I}(S)} \sum_{w \in C_i} \mathrm{PageRank}(w) \\ v_p \in C^*, \quad \text{s.t.} \quad \mathrm{PageRank}(v_p) = \max_{v \in C^*} \mathrm{PageRank}(v) \end{cases} \tag{17}$$

Here, $\mathcal{C}$ is the set of all communities, and $\mathcal{I}(S)$ denotes the communities that have already been covered by S.
(4) **Solution update.** The refined solution S' is generated through replacing v_s with v_p according to Eq. (18).

$$S' = (S \setminus \{v_s\}) \cup \{v_p\} \tag{18}$$

3.4.3. The VNS procedure

The execution logic of the VNS strategy centers on the iterative refinement of S (initialized as G_{best}), as detailed in Alg. 4. The workflow is structured into four pivotal stages.

Shaking. Apply the mth operator $\mathcal{L}_m$ in Section 3.4.2 to S to generate a perturbed solution S'.

Local search. To ensure S' reaches its full potential, a local search follows to refine it within its immediate neighborhood to intensify the search to rapidly converge to a new, potentially higher-quality solution. First, nodes in S' are ranked in descending order by $Score$ defined in Eq. (10) to yield the sequence $v_1, v_2, \ldots, v_k$. Next, for each node v_i, the algorithm examines its direct neighbors $N_1(v_i)$ and selects the top cn nodes with the highest $Score$ to form a candidate set $\mathcal{CA}_i$. Then, a new solution S'' is formed by replacing v_i with a node $u \in \mathcal{CA}_i$, defined as Eq. (19).

$$S'' = (S' \setminus \{v_i\}) \cup \{u\}, \quad u \in \mathcal{CA}_i \tag{19}$$

The refined solution S'' is retained only if it surpasses the original input S'.

Solution acceptance. The transition from S to S'' is governed by a strict improvement rule. Only when $LIE(S'') > LIE(S')$ is the incumbent solution updated. Such a monotonic acceptance rule safeguards against quality degradation evoked by randomness and ensures stable convergence.

Adaptive neighborhood change step and termination. The neighborhood change step adapts to the evolutionary state of the ABC to balance search depth and computational cost.

- *Sequential mode* (Set boolean flag $cyclic$ = False): Prioritizes efficiency. This mode is preferred when the global best solution is steadily evolving to save costs. If improved, the process exits immediately (Line 18 in Alg. 4 as per the "first improvement" strategy) and returns the refined S. Conversely, if no improvement occurs, the search proceeds to the next neighborhood structure by incrementing m to expand the search radius. The VNS concludes immediately upon an improvement being successfully captured or the neighborhood index m exceeds the maximum tier $m_{\max}$ without any improvement.
- *Cyclic mode* (Set boolean flag $cyclic$ = True): Prioritizes thoroughness. Activated during the stagnation stage, this mode ensures an exhaustive traversal of all defined neighborhood structures ($\mathcal{L}_1 - \mathcal{L}_3$) to maximize the probability of escaping local optima. Even if an improvement is found early, the process proceeds to the next neighborhood tier ($m \leftarrow m + 1$). S is updated whenever an improvement is found, but termination occurs only when the index m exceeds $m_{\max}$.

To provide a clearer view of the execution flow, the above logic is summarized in the following step-wise procedure.

Step 1: Initialize the neighborhood index $m = 1$. Set the improvement flag $found \leftarrow$ False. Execute Step 2.

Step 2: Apply the mth operator $\mathcal{L}_m$ to the current solution S to generate a perturbed solution S'. Execute Step 3.

Step 3: Rank all nodes in S' in descending order by their $Score$ (Eq. (10)) to yield the sequence $\{v_1, v_2, \ldots, v_k\}$. Initialize the node counter $i = 1$. Execute Step 4.

Step 4: For the node v_i, examine its direct neighbors $N_1(v_i)$ and select the top cn nodes with the highest $Score$ to construct a candidate set $\mathcal{CA}_i$. Execute Step 5.

Step 5: Select an unexplored candidate u from $\mathcal{CA}_i$. If all candidates have been explored, set $i = i + 1$ and execute Step 7. Generate a new solution S'' by replacing v_i with u and execute Step 6.

Step 6: If $LIE(S'') > LIE(S')$, set $found \leftarrow$ True, increment $i = i + 1$, terminate the search for the current node v_i, and execute Step 7. If $LIE(S'') \leq LIE(S')$, execute Step 5.

Step 7: If $i \leq k$, return Step 4 to process the next node v_i. If $i > k$, execute Step 8.

Step 8: If the boolean flag $cyclic$ is False and the improvement flag $found$ is True, terminate the VNS procedure immediately and return the updated S. If $cyclic$ is False and $found$ is False, increment the neighborhood index $m = m + 1$. If $cyclic$ is True, whether $found$ is True or False, increment the neighborhood index $m = m + 1$. Execute Step 9.

Step 9: If $m > m_{\max}$, terminate the VNS procedure and output the final solution S. Otherwise, return to Step 2.

In summary, the proposed VNS strategy acts as an independent plugin that is embedded into the ABC framework, designed to achieve a good trade-off between exploration and exploitation. Based on the theoretical principle that a local optimum in one neighborhood is not necessarily a local optimum in another, the mechanism designs a three-tier progressive neighborhood structure ($\mathcal{L}_1$, $\mathcal{L}_2$, $\mathcal{L}_3$), which expands the exploration range of the solution space by increasing the search radius. These larger perturbation radii facilitate the escape from the current attraction basin into unexplored regions. Subsequently, a local search operator is employed for deep exploitation to accelerate the convergence. Furthermore, the adaptive neighborhood switching steps adjust the mode based on the evolutionary state of the ABC to balance search depth and computational overhead. During periods of continuous improvement in the global best solution, the sequential mode is adopted following the first improvement principle to conserve computational overhead. In contrast, during stagnation, the mode switches to cyclic to traverse all neighborhoods, which enhances the probability of escaping local traps. Such a theoretical framework coordinates the dynamic balance between global exploration and local exploitation, thereby achieving a balance between computational cost and solution quality. The effectiveness of the design is validated in Section 4.3.

Alg. 4: Variable neighborhood search(G, S, $m_{\max}$, cn, $cyclic$)

Input: Graph $G = (V, E)$; Initial solution S; Max neighborhood tiers $m_{\max}$; Candidate set size cn; Boolean flag $cyclic$ (`True`: cyclic change mode, `False`: sequential change mode).

Output: Improved solution S

```
Neighborhood tier m ← 1 and Improvement flag found ← False;
while m ≤ m_max do
    // Phase 1: Shaking
    S' ← Apply m-th operator in Section 3.4.2 on S ;
    // Phase 2: Local search
    sorted ← Sort nodes v_i ∈ S' by Score (see Eq. (10)) ;
    foreach node v_i ∈ sorted do
        N_1(v_i) ← Direct neighbors of v_i in G ;
        CA_i ← Select top-cn nodes ∈ N_1(v_i) \ S' by Score ;
        foreach u ∈ CA_i do
            Generate a new solution S'' via Eq. (19) ;
            if LIE(S'') > LIE(S') then
                S' ← S'' ; found ← True ;
                break ;
        if found then
            break;
    // Phase 3: Acceptance criteria
    if LIE(S') > LIE(S) then
        S ← S' ;
        if cyclic ==False then
            break ;        // Sequential mode: exit on improvement
        else
            m ← m + 1 ;    // Cyclic mode: move to next tier
    else
        m ← m + 1 ; // No improvement: increment tier
return S
```

3.5. Population management

While the VNS strategy focuses on the refinement of individual solutions, it does not inherently prevent the decline of population diversity. Pronounced convergence stagnation is liable to emerge in later iterations, manifested by the global best solution remaining unchanged for prolonged periods and a sharp decline in population diversity. Such a phenomenon arises as exploration weakens and the search space narrows, forcing solutions to converge toward similar structures. Although ABC hinges on scout bees to promote exploration, the mechanism fails to resolve the issue since it is triggered only after a fixed number of unsuccessful trials.

In light of this, a stagnation-aware population management mechanism is devised via real-time monitoring of the evolution of the global best solution. Once the stagnation counter of the global best solution (τ) exceeds the threshold ($\tau_{\max}$), the mechanism summarized in Alg. 5 is performed to unlock routes to transcend local optima. Firstly, the top $r \cdot NP$ individuals ranked by fitness are retained to preserve elite solutions. Secondly, the remainder is reinitialized by the previously defined Score-guided approach in Section 3.4. Subsequently, the VNS strategy systematically refines the best solution in the current population P_{best} to revive the search using a more exhaustive *cyclic change step*. Note that, as introduced in Section 3.4, the *sequential change step* is preferred when the algorithm is still making consistent progress and solution quality is steadily improving. And the *cyclic change step* is activated whenever the search stalls to ensure that no potential neighborhood is neglected, thereby maximizing the chances of reviving the search.

The stagnation-aware population management mechanism functionally complements the bifurcated scout bee strategy in Section 3.3.4. The former intervenes in convergence stagnation from a population-wide perspective to maintain search vitality, while the latter guides the construction of diverse high-quality solutions at the individual level to enrich population vitality. The impact of the stagnation-aware population management mechanism is examined in Section 4.3, and the effectiveness of the *cyclic change step* in the VNS strategy during stagnation is empirically validated.

3.6. Time complexity analysis

The time complexity of the ABCVNS algorithm is primarily determined by seven principal parameters: g_{max}, NP, L, τ_{max}, ts, r, and cn. Let k denote the seed set size, $\bar{D}$ the average degree of the network, and $|V|, |E|$ the number of nodes and edges, respectively. Firstly, the time complexity of calculating the fitness function LIE is $O(k\bar{D}^2)$. In the initialization phase, the algorithm calculates node degrees and

Alg. 5: Population management(G, P, G_{best}, τ, $\tau_{\max}$, r, TM, $m_{\max}$, cn)

Input: Graph $G = (V, E)$; Population P of size NP; The global best solution G_{best}; Stagnation counter τ; Stagnation threshold $\tau_{\max}$; Retention ratio r; Trial counters TM; Max neighborhood tiers $m_{\max}$; Candidate set size cn.

Output: Population P, global best solution G_{best}, trial counters TM and stagnation counter τ.

if $\tau \geq \tau_{\max}$ **then**
  Sort P in descending order of fitness ;
  $P_{\text{elite}} \leftarrow$ Select the top $r \cdot NP$ individuals from P ;
  $P' \leftarrow P_{\text{elite}}$;
  foreach $S_i \in P \setminus P_{elite}$ **do**
    Generate a new solution S_i' by sampling based on Eq. (10) ;
    $S_i \leftarrow S_i'$;
    $TM_i \leftarrow 0$;
    $P' \leftarrow P' \cup \{S_i\}$;
  $P \leftarrow P'$;
  $P_{\text{best}} \leftarrow$ The best individual in the population P ;
  // Apply the VNS strategy with a cyclic neighborhood change step
  $S \leftarrow$ Variable neighborhood search($G, P_{\text{best}}, m_{\max}, cn,$ `True`) ;
  if $LIE(S) > LIE(G_{best})$ **then**
    $G_{\text{best}} \leftarrow S$
  $\tau \leftarrow 0$; // Reset stagnation counter
return P, G_{best}, TM, τ

performs sorting with a complexity of $O(|V|+|E|+|V|\log|V|)$, followed by the generation and evaluation of NP initial individuals requiring $O(NP \cdot k\bar{D}^2)$ time. Thus, the total complexity of the initialization phase is $O(|V|\log|V| + NP \cdot k\bar{D}^2)$. During the iterative process, the employed bee stage updates NP solutions with a complexity of $O(NP \cdot k\bar{D}^2)$. Subsequently, the onlooker bee stage performs tournament selection of size ts and fitness re-evaluations for NP individuals, contributing $O(NP(ts + k\bar{D}^2))$. In the scout bee stage, the worst-case scenario occurs when all food sources reach the abandonment limit L, leading to a complexity of $O(NP \cdot k\bar{D}^2)$. The VNS optimization phase identifies community structures with a complexity of $O(|V| + |E|)$ and explores $m_{\max}$ neighborhood structures, where a local search evaluates cn candidate replacements for each of the k seed nodes. Since each replacement check requires an LIE evaluation, the complexity of the VNS phase per iteration is $O(m_{\max} \cdot k^2 \cdot cn \cdot \bar{D}^2)$. Additionally, a stagnation-aware population management mechanism triggered by the τ_{max} threshold involves sorting the population and re-evaluating $NP(1-r)$ individuals with a complexity of $O(NP\log NP + NP \cdot k\bar{D}^2)$. Summing up the above analysis, the final time complexity of the ABCVNS is expressed as $T_{total} = O(g_{max} \cdot k \cdot \bar{D}^2 \cdot (NP + m_{\max} \cdot k \cdot cn))$.

4. Experimental results and analysis

This section systematically evaluates ABCVNS that covers experimental settings, module-wise ablation studies, and comparative analyses against up-to-date algorithms. All procedures were encoded in Python and executed on a Windows 11 server equipped with an Intel(R) Core(TM) i9-14900HX processor (2.2 GHz) and 32 GB of RAM.

4.1. Experimental settings

Datasets. Nine real-world network datasets with diverse domains and structures are utilized to validate the algorithm's effectiveness and robustness. The topological properties are summarized in Table 1. All datasets are publicly available and can be freely downloaded from https://www.example.com and https://networkrepository.com/index.php.

Baselines. A rigorous demonstration of the superiority and robustness of the proposed algorithm is achieved by comparisons with a comprehensive suite of baselines ranging from classical greedy approximations to recent state-of-the-art meta-heuristics. The selection rationales are as follows: (1) CELF [14], renowned for a classic greedy algorithm with a theoretical guarantee, serves as a benchmark for accuracy to evaluate the solution quality of the proposed algorithm. (2) DCGM+ [22], a representative high-efficiency heuristic, is selected to demonstrate that the proposed algorithm achieves superior solution quality without sacrificing computational speed. (3) LIDDE [28], an evolutionary algorithm, is included to assess performance against competitive meta-heuristics. (4) PHEE [29], a recent hybrid competitor that integrates simulated annealing to refine search outcomes, is chosen to test the proposed algorithm's effectiveness. (5) ADABC [33], the most advanced ABC variant, is used to verify the specific improvements of our proposed mechanisms over existing ABC frameworks. (6) MFEA-RCIM [38], a pioneering multifactorial evolutionary framework for the IM variants, is included to evaluate whether the proposed algorithm can achieve competitive performance compared to advanced optimization paradigms that utilize inter-task knowledge transfer and latent task synergies. (7) MFEA-RCIM$_{\text{MD}}$ [39], a state-of-the-art extension featuring multi-phase transfer strategies, is selected to benchmark the proposed algorithm's performance and its search efficiency in handling large-scale and complex search spaces. For a fair comparison under a unified problem setting, methods originally designed for more complex variants of IM are adapted to the IM by aligning their objective functions, while preserving their core optimization mechanisms.

Parameters configuration. The ABCVNS algorithm operates under seven principal parameters, namely maximum iterations $g_{\max}$, population size NP, abandonment limit for food sources L, stagnation threshold $\tau_{\max}$, tournament size ts, retention ratio r, and local search evaluation count cn. Optimal parameter values in the study were identified via an orthogonal experimental design in Section 4.2. Furthermore, for fairness, all comparative algorithms strictly adhere to the parameter settings and technical implementation details prescribed by their original authors. Each method is exhaustively executed 30 times for reliability, and the reported results correspond to the average performance across runs.

4.2. Orthogonal experiments

Since the number of levels for each parameter is large, designing all possible experimental combinations would result in an enormous workload. Therefore, an orthogonal experimental design approach is adopted. Systematically selecting representative factor combinations through orthogonal arrays reduces the number of experiments while still obtaining sufficient data and ensuring the reliability of the results. As discussed above, the ABCVNS has seven key parameters. Following MOABC [34], L and $\tau_{\max}$ are fixed at 5 to align with the stagnation-based stopping criterion. The remaining five parameters were tuned using an $L_{18}(3^7)$ orthogonal array. Specifically, the three candidate levels for these parameters are set as: $g_{\max} \in \{100, 150, 200\}$, $NP \in \{10, 50, 100\}$, $ts \in \{2, 3, 5\}$, $r \in \{0.2, 0.35, 0.5\}$, and $cn \in \{5, 10, 15\}$. Eighteen configurations were evaluated 50 times on six representative real-world networks, including Blog, CA-GrQc, Lastfm, CA-HepTh, PGP, and NetHEHT, under $k = 100$ and $p = 0.05$, with average influence spread (μ) and standard deviation (σ) recorded in Table 2.

Table 2 indicates that the parameter combination No. 11 yields the strongest influence spread on the Blog, CA-GrQc, CA-HepTh, and Deezer networks, whereas No. 4 performs as the optimal merely on the Lastfm network, and No. 15 acts as the best on the NetHEHT network. The observation that different parameter configurations excel on different network types aligns with the No Free Lunch Theorem, which posits that no single setting is universally superior. While larger

Table 1
Datasets description.

ID	Datasets	Type	$\|V\|$	$\|E\|$	$k_{\max}$	$\langle k\rangle$	$\langle c\rangle$	r
1	Blog	Social	3982	6803	189	3.42	0.28	−0.13
2	CA-GrQc	Collaboration	5242	14 496	81	5.53	0.53	0.66
3	Lastfm	Social	7624	27 806	216	7.29	0.22	0.02
4	CA-HepTh	Collaboration	9877	25 998	65	5.26	0.47	0.27
5	Twin	Social	14 274	20 572	99	2.88	0.02	0.02
6	NetHEHT	Collaboration	15 229	31 376	64	4.12	0.50	0.39
7	Deezer	Social	28 281	92 752	172	6.56	0.14	0.10
8	Soc-slashdot	Social	70 069	358 647	2507	10.24	0.03	−0.05
9	Soc-delicious	Social	536 109	1 365 961	3216	5.10	0.03	−0.07

Note: $|V|$, $|E|$, $k_{\max}$, $\langle k\rangle$, $\langle c\rangle$, and r denote the number of vertices, the number of edges, the maximum degree, the average degree of nodes, the clustering coefficient, and the assortativity coefficient, respectively.

Table 2
The mean value (μ) and the standard deviation (σ) of the proposed algorithm under distinct parameter settings on the six datasets. Best results are indicated in bold.

No.	$g_{\max}$	NP	ts	r	cn	Blog ($\mu \pm \sigma$)	CA-GrQc ($\mu \pm \sigma$)	Lastfm ($\mu \pm \sigma$)	CA-HepTh ($\mu \pm \sigma$)	NetHEHT ($\mu \pm \sigma$)	Deezer ($\mu \pm \sigma$)
1	100	10	2	0.2	5	255.48 ± 1.28	328.14 ± 2.65	703.46 ± 2.49	396.51 ± 1.54	372.32 ± 1.55	972.81 ± 2.23
2	100	50	3	0.35	10	255.61 ± 0.66	330.18 ± 2.16	702.68 ± 1.99	397.97 ± 1.32	374.12 ± 1.86	975.28 ± 1.99
3	100	100	5	0.5	15	256.13 ± 0.97	331.19 ± 2.32	704.17 ± 2.42	399.15 ± 1.26	375.98 ± 1.57	977.61 ± 1.51
4	100	10	2	0.35	10	254.27 ± 1.14	327.67 ± 1.11	**706.56 ± 0.46**	398.13 ± 1.35	376.54 ± 1.43	976.57 ± 1.64
5	100	50	3	0.5	15	256.47 ± 0.38	330.96 ± 0.82	703.73 ± 2.75	397.95 ± 0.99	373.95 ± 1.92	975.76 ± 1.27
6	100	100	5	0.2	5	254.53 ± 0.72	327.43 ± 0.93	704.11 ± 2.63	395.81 ± 1.54	372.10 ± 1.24	974.10 ± 1.06
7	150	10	3	0.2	15	255.86 ± 0.81	330.85 ± 0.95	703.47 ± 1.72	396.55 ± 0.99	374.35 ± 1.16	977.34 ± 1.58
8	150	50	5	0.35	5	255.61 ± 1.21	331.56 ± 1.01	703.53 ± 1.81	397.73 ± 1.02	373.45 ± 0.99	980.45 ± 1.44
9	150	100	2	0.5	10	255.72 ± 0.92	333.36 ± 1.23	704.97 ± 0.99	398.17 ± 0.87	375.28 ± 1.03	977.31 ± 1.13
10	150	10	5	0.5	5	254.77 ± 1.38	329.52 ± 1.29	705.32 ± 1.24	397.85 ± 0.72	372.75 ± 0.94	979.92 ± 1.02
11	150	50	3	0.2	10	**256.85 ± 0.16**	**334.04 ± 0.34**	704.56 ± 1.51	**399.78 ± 0.29**	376.68 ± 0.55	**981.11 ± 0.41**
12	150	100	2	0.35	15	256.03 ± 1.31	328.36 ± 1.31	705.94 ± 1.26	398.14 ± 1.06	376.52 ± 0.69	980.53 ± 0.95
13	200	10	5	0.35	15	255.64 ± 0.72	327.85 ± 0.98	705.58 ± 0.93	396.42 ± 1.07	375.22 ± 0.58	979.91 ± 0.84
14	200	50	2	0.5	5	256.38 ± 0.86	328.73 ± 1.17	704.62 ± 0.95	397.83 ± 1.08	375.56 ± 0.79	976.27 ± 0.89
15	200	100	3	0.2	10	255.95 ± 1.08	330.22 ± 1.34	704.91 ± 0.98	399.65 ± 1.09	**376.84 ± 0.32**	977.80 ± 0.73
16	200	10	3	0.5	10	256.62 ± 0.56	328.05 ± 1.03	705.12 ± 0.91	398.78 ± 1.14	373.56 ± 0.51	978.12 ± 0.66
17	200	50	5	0.2	15	255.97 ± 0.94	331.71 ± 0.72	706.43 ± 0.74	398.25 ± 1.11	375.44 ± 0.47	978.53 ± 0.91
18	200	100	2	0.35	5	255.41 ± 1.13	332.85 ± 0.64	706.38 ± 0.53	397.49 ± 1.13	372.24 ± 0.39	979.76 ± 0.74

population size and more iterations raise the probability of reaching the global optimum, they also lead to higher time complexity. Therefore, based on a comprehensive assessment of the orthogonal experiments, the final parameter settings are determined to balance performance and efficiency. Specifically, $g_{\max} = 150$ ensures the convergence of ABCVNS. $NP = 50$ maintains sufficient population diversity. $ts = 3$ prevents exclusive selection pressure. $r = 0.2$ preserves elite solutions while preventing premature convergence. And $cn = 10$ controls the search depth and strikes a well-balanced trade-off between efficiency and effectiveness.

Although the orthogonal experimental design in Section 4.2 identified an optimal parameter combination, it is essential to investigate the sensitivity of the proposed algorithm to key parameters to ensure its robustness. Therefore, a systematic sensitivity analysis focusing on two pivotal parameters, i.e., number of iterations ($g_{\max}$) and population size (NP), is performed. Such an analysis provides insights into the trade-off between solution quality and computational overhead, ensuring the robustness of the algorithm across diverse network topologies. To isolate the impact of each parameter, the control variable method is adopted here, wherein the other three parameters remain fixed at their optimal values derived from the orthogonal experiments. The number of iterations was varied across three levels $g_{\max} \in \{100, 150, 200\}$, and the population size across three levels $NP \in \{10, 50, 100\}$. Each configuration was independently run 10 times on the same six networks used in the orthogonal experiments to mitigate stochastic fluctuations.

Regarding the maximum number of iterations $g_{\max}$, Fig. 2 presents the fitness values obtained across the evaluated datasets. A consistent upward trend in fitness is observed as $g_{\max}$ increases from 100 to 150 across all networks. Such an improvement indicates that a limited number of iterations is insufficient for the ABC-VNS to adequately explore the solution space. However, as $g_{\max}$ further expands from 150 to 200, the incremental gains in fitness become marginal, with a visible plateau emerging in most datasets, particularly for CA-GrQc and Twin. Given that the increased computational overhead incurred by $g_{\max} = 200$ does not yield a significant performance enhancement, $g_{\max} = 150$ is identified as the most cost-effective configuration.

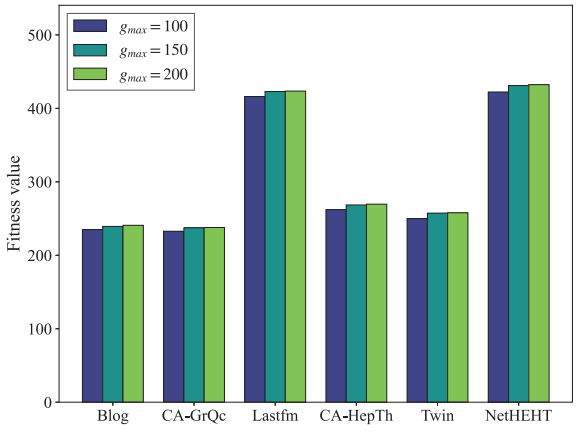


Fig. 2. The parameter sensitivity analysis of the number of iterations $g_{\max}$.

The parameter NP serves as a critical determinant of the balance between global exploration and local exploitation within population-based metaheuristics. Fig. 3 presents the statistical distribution of the

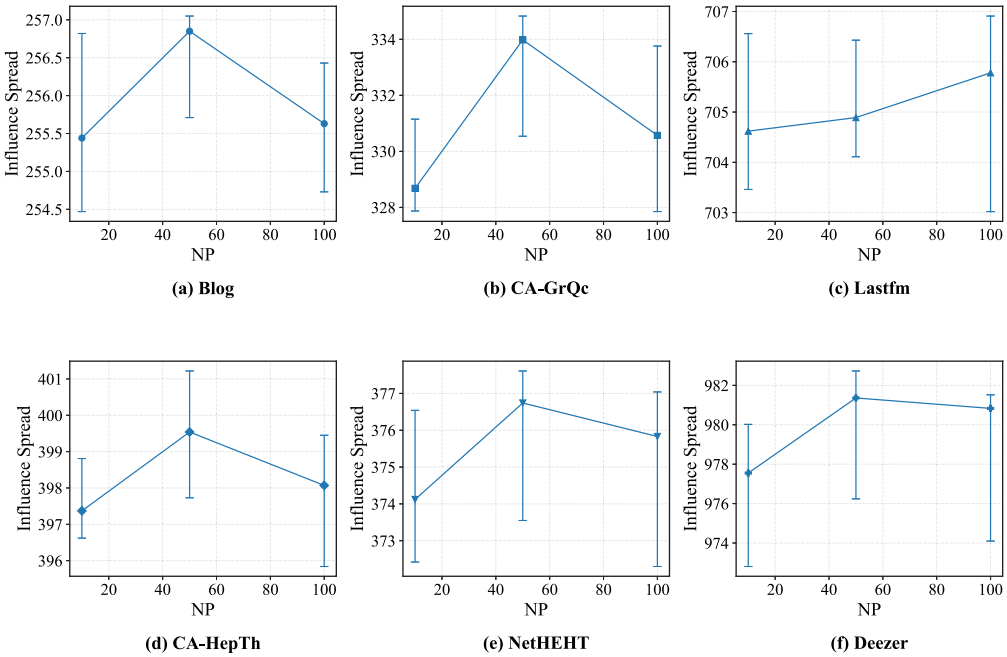


Fig. 3. The parameter sensitivity analysis of the population size NP.

influence spread for varying NP levels, where the minimum, maximum, and average values for each configuration are recorded. For most networks (Blog, CA-GrQc, CA-HepTh, NetHEHT, Deezer), the average influence spread reaches its peak at $NP = 50$, followed by a marginal decline or plateau as NP increases to 100. Such a phenomenon suggests that a medium-sized population is more conducive to concentrating the search on high-quality solution regions while maintaining sufficient exploration capability.

Specifically, in datasets such as Blog and CA-HepTh, the transition from $NP = 10$ to $NP = 50$ yields a substantial improvement in the final influence spread, highlighting the necessity of maintaining sufficient population diversity. However, further expansion of the population size from 50 to 100 does not result in a proportional enhancement in performance. In several cases, including the Blog and NetHEHT datasets, a marginal decrease in the average influence spread is observed at $NP = 100$. Such a trend is indicative of the critical trade-off between exploration breadth and exploitation intensity. In contrast, the Lastfm dataset exhibits a continued increase in influence spread at $NP = 100$, yet the gap between the maximum and minimum values is notably large. These results imply that larger populations explore a wider solution space in these networks, while also exhibiting increased variability in the obtained influence spread. In summary, the sensitivity analysis demonstrates that $NP = 50$ provides a superior trade-off between search quality and result stability across the diverse set of networks.

4.3. Module-wise performance evaluation

The ablation framework herein isolated the effects of three pivotal innovations: VNS, EGCNC, and stagnation detection. Corresponding variants, i.e., VNS-Disabled, EGCNC-Disabled, and Stag-Disabled, were derived from ABCVNS. A further variant, StagSwitch, compared sequential versus cyclic neighborhood switching when progress ceases. Each underwent 30 independent trials on the above six networks, and the average influence spread for $k \in [10, 100]$ served as an indicator of performance degradation. Fig. 4 depicts the relative performance of each ablation variant compared to the full ABCVNS model across these networks. Specifically, the full ABCVNS model is designated as the baseline with a normalized score of 1.0, and the performance of each ablation variant is normalized accordingly. Table 3 reports the detailed average influence spread for ABCVNS and its four ablation variants. The rationale and contributions of each feature are analyzed below.

VNS-disabled. To isolate the contribution of the VNS component to the algorithm's local search and exploitation capabilities, a VNS-Disabled variant was developed for comparison. The results in Fig. 4 demonstrate that ABC-VNS consistently outperforms the VNS-Disabled version across all six datasets. As illustrated by the relative performance change plots, the removal of the VNS strategy leads to a substantial decline in solution quality. The results in Table 3 indicate that, compared with the influence spread of the VNS-Disabled, ABCVNS has an improvement of approximately 14.9% (Blog), 9.4% (CA-GrQc), 4.31% (Lastfm), 9.4% (CA-HepTh), 13.94% (NetHEHT), and 13.33%(Deezer) when $k = 100$. Such a significant numerical gap underscores a fundamental limitation of single-neighborhood search strategies, which struggle to navigate the multi-modal and rugged landscapes of the IM problem. In contrast, the multi-neighborhood search strategies in ABCVNS enable the algorithm to reach unexplored regions of the solution space, thereby facilitating the escape from local optima. Further, the monotonic acceptance rule ensured that solution quality never degraded, facilitating a robust convergence profile. Consequently, the empirical evidence again suggests that the VNS strategy compensates for the original ABC's shortcomings in exploitation, ultimately boosting solution quality.

EGCNC-disabled. The variant dissects the role of informed perturbations by removing elite guidance in favor of a random node recombination. Fig. 4 shows that ABCVNS consistently outperforms the variant on all datasets. ABCVNS improves the influence spread by approximately 1.08% (Blog), 1.27% (CA-GrQc), 1.63% (Lastfm), 3.3% (CA-HepTh), 2.45% (NetHEHT), and 1.15% (Deezer) at $k = 100$ in Table 3. The impact is particularly evident on the NetHEHT network with $k = 30$, where the influence spread increases from 179.8492 to 195.4778, an 8.69% gain—the largest improvement among the tested configurations. The observed performance decay underscores the role of the EGCNC

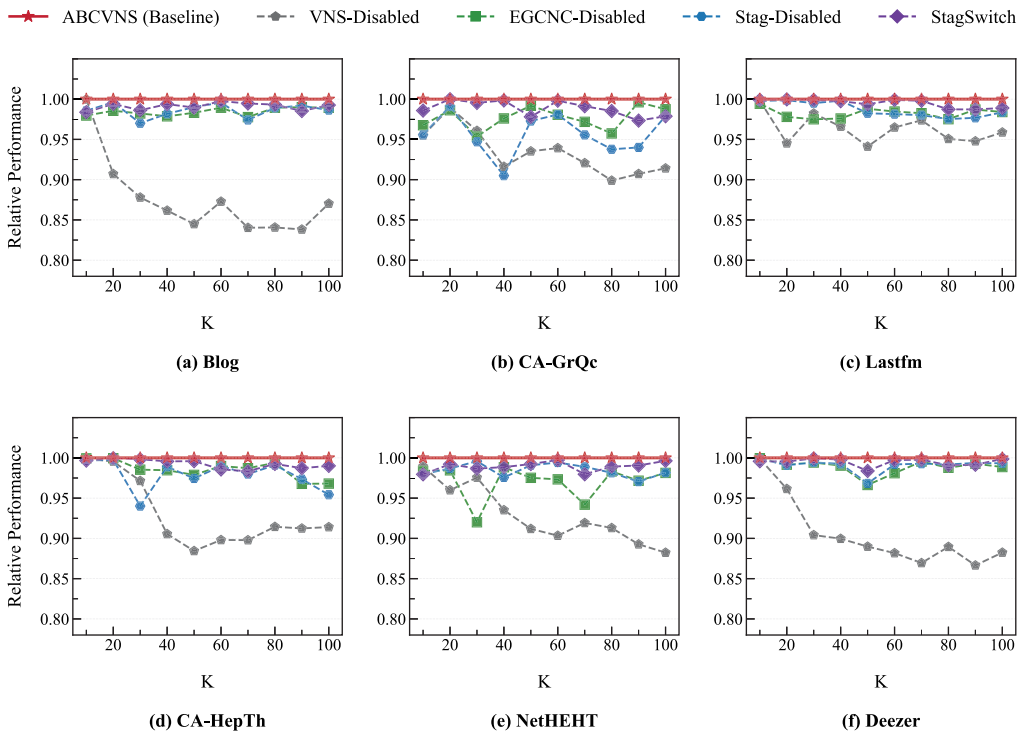


Fig. 4. Relative performance of ablation variants with respect to ABCVNS across six real-world networks.

Table 3
Ablation study results across networks (Average influence spread). Best results are indicated in bold.

Network	Ablation	$k = 10$	$k = 20$	$k = 30$	$k = 40$	$k = 50$	$k = 60$	$k = 70$	$k = 80$	$k = 90$	$k = 100$
Blog	ABCVNS	**54.9068**	**87.4395**	**115.9066**	**141.6812**	**162.7132**	**182.3482**	**204.0938**	**222.8963**	**240.1242**	**256.8531**
	EGCNC-Disabled	53.7889	86.1747	113.8434	138.6610	159.9587	180.4018	199.5358	220.5126	237.7213	254.1075
	VNS-Disabled	54.8869	79.3166	101.7535	122.0767	137.4772	159.1498	171.4953	187.3793	201.2610	223.5378
	Stag-Disabled	54.1148	87.0994	112.4396	139.1017	161.1051	181.5573	198.8171	220.4873	238.1773	253.3644
	StagSwitch	54.0149	86.9211	114.3083	140.7994	161.0143	181.8884	202.9901	221.3795	236.5872	254.9552
CA-GrQc	ABCVNS	**117.8332**	**157.1472**	**190.1216**	**215.8712**	**240.7558**	**260.5826**	**282.2212**	**302.5401**	**317.8852**	**334.0404**
	EGCNC-Disabled	114.0166	154.9809	181.2794	210.6955	238.7636	255.5273	274.2601	289.6516	316.6599	329.8391
	VNS-Disabled	112.5090	155.4926	182.6513	197.7690	225.1640	244.7511	259.8170	271.9449	288.3707	305.3424
	Stag-Disabled	112.6015	155.7803	180.0551	195.3596	234.2129	255.5971	269.6731	283.6954	298.8153	327.0431
	StagSwitch	116.1616	157.1353	189.2468	215.5203	235.4644	260.0721	279.7440	298.0749	309.4130	326.9383
Lastfm	ABCVNS	**429.8843**	**487.4312**	**530.1114**	**560.4550**	**591.3653**	**624.9612**	**642.0030**	**672.2355**	**690.1741**	**704.5587**
	EGCNC-Disabled	427.4252	476.6951	516.8563	546.9655	584.2745	615.3236	630.7873	655.5005	681.2445	693.2758
	VNS-Disabled	429.0796	460.6855	520.6819	541.3515	556.5480	602.9901	625.1450	639.0513	654.1223	675.4733
	Stag-Disabled	428.9410	486.7489	527.4739	559.4096	580.9904	613.3325	629.2148	655.6103	674.1667	692.9620
	StagSwitch	429.4628	487.2829	529.5494	559.0070	587.6128	624.3533	640.6847	663.7019	681.3424	696.7903
CA-HepTh	ABCVNS	**94.3315**	**155.8448**	**196.4561**	**231.0112**	**267.8404**	**297.3199**	**324.0237**	**350.3548**	**379.7024**	**399.7785**
	EGCNC-Disabled	94.2724	155.7781	193.5740	227.4700	262.1726	294.2965	319.8669	348.2002	367.4642	387.0033
	VNS-Disabled	94.2150	155.1506	190.8452	209.1696	236.8816	267.0077	290.9222	320.3705	346.4355	365.4178
	Stag-Disabled	94.0260	155.4556	184.6973	228.4784	260.9922	294.3132	317.5144	347.1770	369.6659	381.4813
	StagSwitch	94.0174	155.7954	196.1219	230.0050	266.7454	293.0595	318.6425	347.7312	374.8425	395.8135
NetHEHT	ABCVNS	**96.9879**	**158.6759**	**195.4778**	**231.2507**	**257.2050**	**280.6872**	**307.8114**	**331.7992**	**355.2465**	**376.6815**
	EGCNC-Disabled	95.4164	156.2250	179.8492	228.5517	250.8301	273.1978	289.9296	326.2194	345.1601	367.6859
	VNS-Disabled	95.9417	152.3136	190.6917	216.2535	234.5322	253.5723	282.9419	302.9275	317.1176	330.5901
	Stag-Disabled	95.0383	156.5835	194.6688	225.6334	254.8396	279.0733	304.3748	325.6488	344.7682	367.7997
	StagSwitch	95.0057	157.4324	192.8761	228.5919	255.2342	279.9291	301.5413	328.1929	351.8903	373.3029
Deezer	ABCVNS	**377.4962**	**509.8214**	**590.1909**	**672.2498**	**744.9769**	**792.1040**	**846.9892**	**897.2988**	**939.6305**	**981.1134**
	EGCNC-Disabled	377.3150	505.5246	586.7606	665.8063	719.9017	777.0584	843.0536	886.2831	932.7909	969.9650
	VNS-Disabled	377.0622	490.1996	533.6068	604.7448	662.8828	698.4232	736.4208	798.1803	814.2123	865.7350
	Stag-Disabled	377.0977	505.1537	586.8040	667.2798	721.3441	785.6832	840.8468	889.9511	933.6977	974.4814
	StagSwitch	375.9045	507.5192	589.7944	670.8774	732.8850	789.9521	845.0571	889.0014	931.6156	979.7645

mechanism in harnessing structural insights from elites to mitigate aimless exploration.

Stag-disabled. Exclusion of the stagnation detection mechanism from the ABCVNS defines the Stag-Disabled variant. Table 3 reveals that its absence results in a clear and consistent decline in solution quality, with the performance gap reaching a maximum of 10.5% as k increases. Such a pronounced gap stems from increased susceptibility of larger seed sets to stagnation due to diminishing marginal gains. So our detection mechanism effectively disrupts these pseudo-stable states through dynamic updates. Besides, network topology also dictates the mechanism's effectiveness. A notable observation is that at $k = 80$, ABCVNS outperforms

the variant by 6.64% on CA-GrQc but only 0.83% on Deezer. The disparity is rooted in the scale-free nature of CA-GrQc, where edges are highly concentrated within local communities, frequently trapping the search during expansion. In contrast, the abundant cross-community connections in Deezer make it less susceptible to being trapped in dead ends and diminish the relative advantage.

StagSwitch. The default *cyclic change step* in the population management mechanism was modified into a *sequential change step* to construct the StagSwitch variant to evaluate its impact when search progress stalls. As shown in Fig. 4, the performance gap between ABCVNS and StagSwitch is negligible, with an average improvement of only 0.82% across all networks. The runtime efficiency, however, increased by 11.21%, a gain primarily attributed to the reduced complexity of sequential switching by eliminating exhaustive traversals. The preservation of core local search and directional guidance ensures equivalent performance and allows the algorithm to benefit from lower computational costs without sacrificing solution quality.

4.4. Comparative analysis

4.4.1. Performance on influence spread

Influence diffusion was traced across varying seed set sizes to shed light on propagation dynamics. To ground the observation in quantitative terms, the influence spread for $k \in [10, 100]$ was estimated via 1000 Monte Carlo simulations, with values averaged over 30 independent runs per algorithm. Experimental configurations included an edge activation probability of $p = 0.05$ under the IC model for the seven smaller networks (Blog, CA-GrQc, Lastfm, CA-HepTh, Twin, NetHEHT, Deezer), while a probability of $p = 0.01$ was adopted for the two relatively large-scale networks, Soc-slashdot and Soc-delicious, to mitigate computational overhead. Community structures were extracted through the *Louvain* algorithm [40]. Section 4.1 outlines the selection rationale and characteristics of the compared algorithms. Finally, Fig. 5 presents the results of eight algorithms on nine networks, where the x and y axes denote seed size and influence spread, respectively.

Analysis of the results in Fig. 5 clearly shows that the proposed algorithm slightly underperforms CELF but surpasses the other algorithms. In Figs. 5(a), (d), (e), (g), (h), and (i), ABCVNS's performance rivals that of the greedy algorithm. The high accuracy benefits from the well-grained VNS strategy that prevents the algorithm from getting stuck in local optima within a single neighborhood. In Fig. 5(b), the performance gap is observed in the CA-GrQc network at $k = 100$, with a difference of 9.1% due to the network's relatively high assortativity. Nodes in such a high assortativity network tend to connect with others of similar degree, limiting the diversity of reachable neighbors and making exploration more difficult. In contrast, DCGM+ performs the weakest, ranking last in all networks with a noticeably smaller influence spread, and its gains diminish as the seed set size increases. ABCVNS improves influence spread by approximately 40.67% (Blog), 64.49% (CA-GrQc), 27.91% (Lastfm), 26.56% (CA-HepTh), 25.63% (Twin), 18.18% (NetHEHT), 16.66% (Deezer), 8.11% (Soc-slashdot), and 18.71% (Soc-delicious) on average compared to DCGM+. The shortcoming arises primarily because heuristics directly select the top-ranked as seeds, overlooking the influence overlap between seeds. A closer inspection reveals notable differences in how each algorithm scales with seed size. Across all networks, the influence spread of eight algorithms generally increases with larger seed sets, confirming that additional seed nodes facilitate broader diffusion. However, regarding DCGM+, as k grows, the influence overlap between newly added seeds and existing nodes can undermine the overall performance.

In Figs. 5(a), (c), (f), (g), and (i), LIDDE and ADABC exhibit comparable performance, while PHEE achieves superior results. The former's reliance on a fixed-size candidate pool insufficiently adapts to network scale and seed set size. In comparison, owing to the fact that its dynamical adjustments of the candidate pool, PHEE leads to better performance relative to the fixed-pool counterpart. But the reliance on k-core values hinders accurate assessment of node influence. As a result, the construction of the candidate node pool can be compromised. ABCVNS addresses the above limitation by leveraging network structural information to construct the candidate pool. Such a design improves the robustness of the algorithm by reducing the impact of inaccurate candidate node selection. Relative to existing meta-heuristics, ABCVNS demonstrates an increase of 3.39% on Blog, 8.39% on CA-GrQc, 4% on Lastfm, 4.61% on CA-HepTh, 4.84% on Twin, 4.42% on NetHEHT, 2.67% on Deezer, 2.61% on Soc-slashdot, and 1.95% Soc-delicious. These experimental results across these diverse network structural characteristics demonstrate the effectiveness and robustness of our algorithm.

The correlation between network topology and algorithmic performance can be observed across the tested datasets. In networks exhibiting high degree heterogeneity, such as Soc-delicious, where the maximum degree exceeds 3200, the presence of hub nodes tends to bias heuristic strategies toward selecting high-degree nodes. It often results in a concentration of seeds within densely connected regions, leading to substantial influence overlap and consequently limiting marginal gain, which increases the risk of convergence to local optima. The incorporation of VNS within ABCVNS alleviates such a bias by introducing structural diversity into the search process, thereby reducing the dependence on hub-dominated regions.

In addition, structural properties such as assortativity and clustering coefficients further affect the search landscape. Networks with high clustering coefficient and positive assortativity, such as CA-GrQc, typically exhibit strong community structure, where high-degree nodes are more likely to connect with each other. Such a structural pattern restricts diffusion paths to intra-community regions and constrains the propagation toward peripheral nodes. The use of neighborhood structures within the VNS strategy supports transitions across different structural regions, allowing candidate solutions to explore nodes beyond densely connected clusters. Conversely, in networks with relatively homogeneous degree distributions and low average degree, such as Twin, the absence of prominent hubs results in a more uniform distribution of influence potential across nodes. Under such conditions, the marginal contribution of individual nodes is limited, and the influence spread depends more on the joint effect of node combinations. Consequently, the search process requires a balanced coordination between exploration and exploitation, where multiple candidate combinations need to be evaluated to capture the interaction effects among nodes that govern the diffusion process. These observations are consistent with the experimental results reported in Fig. 5, where ABCVNS demonstrates more stable performance in networks with high degree heterogeneity and clustering.

4.4.2. Performance on efficiency

This section embarks on a comprehensive comparison of six algorithms in terms of activated node counts, seed set similarity, and runtime across nine datasets. Fig. 6 shows the representative results under $p = 0.05$ and $k = 100$.

Activated node counts. ABCVNS consistently secures the second-best rank across all datasets, only behind CELF, and decisively surpasses other meta-heuristic competitors. Despite its non-greedy architecture, the cohesive algorithmic design empowers it to facilitate superior solution quality via the enhancement of its capacity to escape local optima.

Seed set similarity. The Jaccard coefficient acts as an essential measure for assessing how consistently different algorithms identify influential nodes, defined in Eq. (20).

$$J(S_A, S_B) = \frac{|S_A \bigcap S_A|}{|S_A \bigcup S_B|} \tag{20}$$

where S_A and S_B denote the seed sets of algorithms A and B, respectively. Across all tested datasets, the average Jaccard coefficient

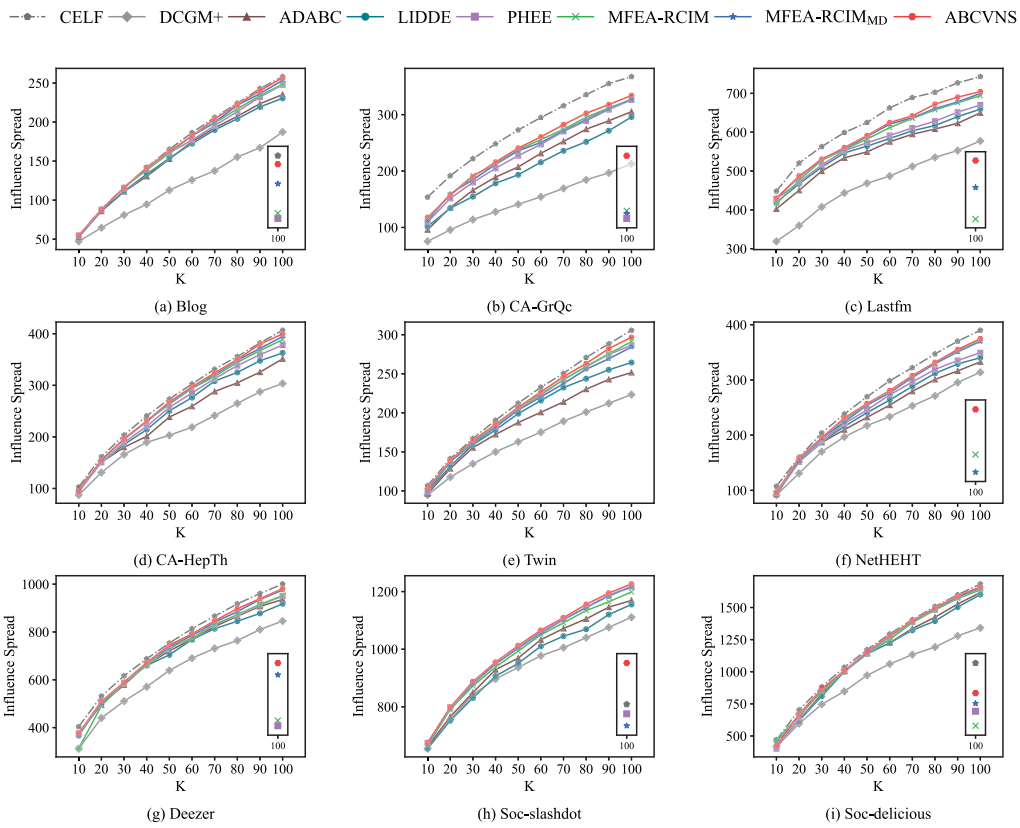


Fig. 5. Comparison of influence spread across nine real networks.

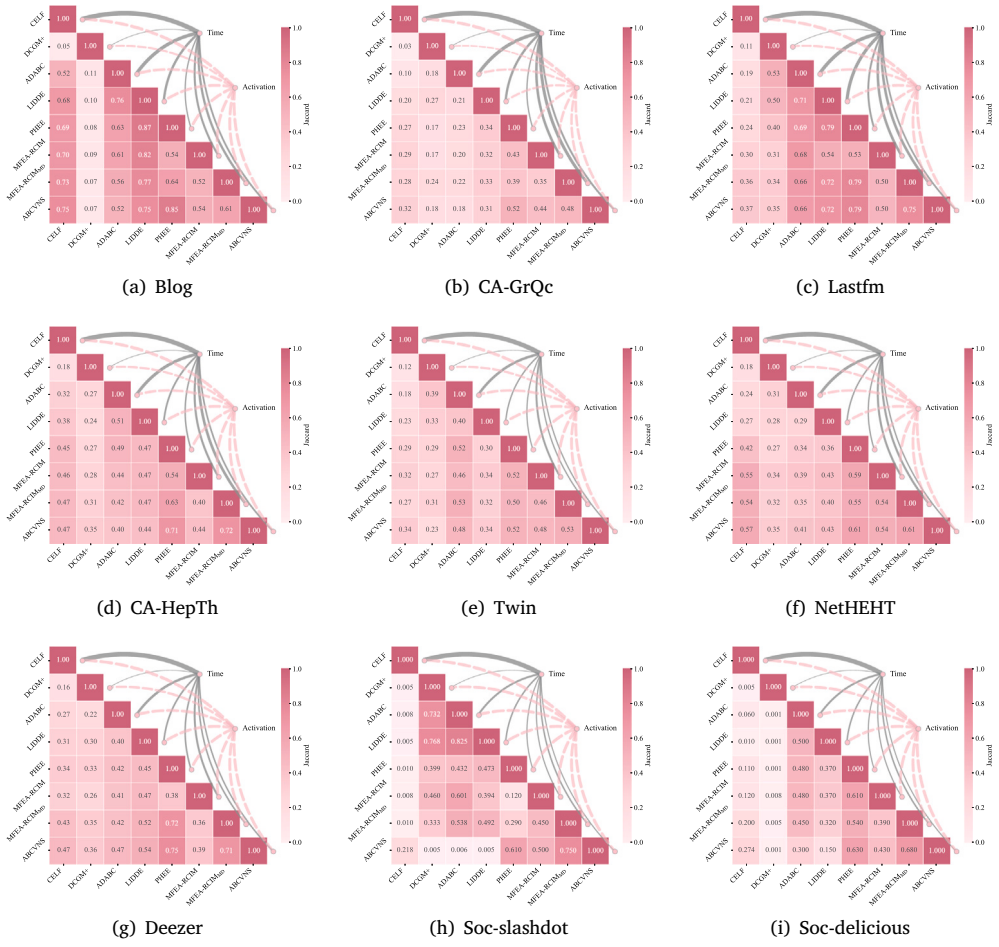
between ABCVNS and DCGM+ is 0.23. The minimal overlap reflects disparate seed compositions and accounts for the inferior performance of DCGM+. Conversely, ABCVNS shares moderate similarity (exceeding 0.5) with ADABC and LIDDE in Figs. 6(a), 6(c), 6(d), 6(e) and 6(g). Grant ed that results are comparable, ABCVNS still maintains a clear advantage on the CA-HepTh network at $k = 100$, with improvements of 13.77% and 10.19%, respectively. Stronger correspondence with PHEE on the Blog network, characterized by a 0.85 Jaccard coefficient, translates to an approximate 3.62% gain in influence spread. Notably, the proposed algorithm consistently achieves the highest structural alignment with CELF and significantly outperforms the other five methods. Such a close proximity to the greedy approach underscores the algorithm's superior capacity to capture critical nodes relative to other methods. These findings collectively validate the robustness and efficacy of the proposed algorithm.

Efficiency and memory overhead. Table 4 summarizes the specific execution time and peak memory consumption of ABCVNS at $k = 100$ across the nine networks. The greedy-based CELF exhibits extremely high time costs on large-scale networks, such as Soc-slashdot and Soc-delicious, reaching hundreds of thousands to over a million seconds. In contrast, the heuristic DCGM+ consumes the least execution time. Its subpar activation performance, as previously analyzed, reaffirms the inherent limitations of heuristics. Its memory usage escalates sharply on large networks (approaching 900 MB for Soc-slashdot and exceeding 3000 MB for Soc-delicious). ABCVNS demonstrates a relatively balanced profile, achieving moderate memory consumption while maintaining computational time substantially lower than meta-heuristics like ADABC and LIDDE, particularly on medium networks such as Deezer and NetHEHT. ADABC suffers from the longest runtime across all meta-heuristics, primarily due to the taxing nature of its evaluation procedure. Other meta-heuristics perform comparably on medium networks but show more variation in time and memory, indicating sensitivity to network scale and topological complexity. Overall, the proposed algorithm demonstrates favorable and stable performance and offers advantages in efficiency.

4.4.3. Statistical tests

To further examine whether the observed performance differences are statistically meaningful, the non-parametric Wilcoxon signed-rank test is conducted at a significance level of $\alpha = 0.05$. The test is performed on nine real-world networks for multiple seed set sizes ($k = 20, 40, 60, 80, 100$), comparing the influence spread achieved by ABCVNS with that of each competing algorithm. Due to the stochastic nature of the algorithms, each method is independently executed 30 times on each network to obtain a reliable estimation of performance. The statistical results are reported in Table 5, where $N+$ and $N-$ represent the number of networks on which ABCVNS achieves higher or lower influence spread than the compared algorithm, respectively. The statistical significance of the observed differences is determined using the corresponding Z values and p-values.

It can be observed that ABCVNS demonstrates statistically significant performance differences against most competing algorithms. For DCGM+, ADABC, and LIDDE, ABCVNS consistently achieves higher influence spread across all nine networks for all seed set sizes ($N- = 0$ and $N+ = 9$). These comparisons yield identical statistical metrics across all values of k, indicating a strong and consistent advantage of ABCVNS over these three methods. In the comparison with PHEE, MFEA-RCIM, and MFEA-RCIM$_{MD}$, ABCVNS also shows a dominant performance, with $N+ = 9$ in most cases. Minor exceptions occur where $N- = 1$, e.g., PHEE at $k = 20$, and MFEA-RCIM at $k = 60$, while the p-values remain below 0.05, demonstrating a stable advantage. For the classical greedy baseline CELF, the results reveal a different pattern. Across all seed sizes, ABCVNS achieves a higher influence spread on



(a) Blog (b) CA-GrQc (c) Lastfm

(d) CA-HepTh (e) Twin (f) NetHEHT

(g) Deezer (h) Soc-slashdot (i) Soc-delicious

Fig. 6. Comprehensive comparison of six algorithms across nine networks in terms of seed set similarity, runtime, and activation ratio.

Table 4
Efficiency comparison across nine datasets.

Datasets	Metric	CELF	DCGM+	ADABC	LIDDE	PHEE	MFEA-RCIM	MFEA-RCIM$_{MD}$	ABCVNS
Blog	Time (s)	7.65E+03	9.91E−01	3.81E+03	1.59E+03	2.51E+02	1.07E+03	1.46E+03	7.43E+02
	Memory (MB)	213.52	105.16	99.53	171.85	93.41	104.51	87.21	125.84
CA-GrQc	Time (s)	9.63E+03	1.34E+00	4.54E+03	1.90E+03	3.24E+02	2.36E+03	3.55E+03	1.11E+03
	Memory (MB)	215.56	108.98	103.04	174.87	101.80	108.19	90.59	138.12
Lastfm	Time (s)	1.67E+04	2.72E+00	8.79E+03	6.99E+03	1.89E+03	4.19E+03	4.32E+03	1.35E+03
	Memory (MB)	246.36	123.53	109.31	181.81	111.86	114.78	97.65	153.97
CA-HepTh	Time (s)	1.54E+04	2.98E+00	5.76E+03	1.47E+03	8.47E+02	2.37E+03	4.26E+03	1.20E+03
	Memory (MB)	243.74	121.05	110.03	181.55	113.08	115.53	98.26	152.52
Twin	Time (s)	2.73E+04	1.35E+00	6.54E+03	1.12E+03	7.08E+02	3.25E+03	9.44E+02	8.64E+02
	Memory (MB)	262.98	125.12	110.99	182.65	114.16	116.54	99.96	164.24
NetHEHT	Time (s)	3.12E+04	3.05E+00	9.18E+03	1.53E+03	1.20E+03	4.53E+03	4.60E+03	1.37E+03
	Memory (MB)	269.56	130.71	115.18	195.71	134.02	120.94	104.22	172.49
Deezer	Time (s)	1.03E+05	6.40E+00	2.25E+04	8.01E+03	6.95E+03	1.28E+04	6.53E+03	5.14E+03
	Memory (MB)	370.29	190.29	145.82	208.875	204.25	153.11	174.37	231.43
Soc-slashdot	Time (s)	6.48E+05	1.66E+01	7.32E+04	3.48E+04	1.28E+04	1.45E+04	3.50E+04	1.08E+04
	Memory (MB)	901.23	890.47	268.94	346.29	316.82	282.39	282.51	500.96
Soc-delicious	Time (s)	1.33E+06	6.21E+01	4.43E+05	2.72E+05	1.56E+05	1.83E+05	3.17E+05	1.22E+05
	Memory (MB)	4004.3	3199.98	935.18	1146.92	1281.63	981.94	947.82	2002.15

Table 5
The statistical results of the Wilcoxon test on ABCVNS and the other seven algorithms at $\alpha = 0.05$ on nine real-world networks.

ABCVNS *vs.*	k	$N-$	$N+$	Z	p-value
CELF	20	8	1	−2.073	0.038
	40	8	1	−2.429	0.015
	60	8	1	−2.192	0.028
	80	8	1	−2.192	0.028
	100	8	1	−2.310	0.021
DCGM+	20	0	9	−2.666	0.008
	40	0	9	−2.666	0.008
	60	0	9	−2.666	0.008
	80	0	9	−2.666	0.008
	100	0	9	−2.666	0.008
ADABC	20	0	9	−2.666	0.008
	40	0	9	−2.666	0.008
	60	0	9	−2.666	0.008
	80	0	9	−2.666	0.008
	100	0	9	−2.666	0.008
LIDDE	20	0	9	−2.666	0.008
	40	0	9	−2.666	0.008
	60	0	9	−2.666	0.008
	80	0	9	−2.666	0.008
	100	0	9	−2.666	0.008
PHEE	20	1	8	−2.547	0.008
	40	0	9	−2.666	0.008
	60	0	9	−2.666	0.008
	80	0	9	−2.666	0.008
	100	0	9	−2.666	0.008
MFEA-RCIM	20	1	8	−2.310	0.021
	40	0	9	−2.666	0.008
	60	1	8	−2.547	0.011
	80	0	9	−2.666	0.008
	100	0	9	−2.666	0.008
MFEA-RCIM$_{MD}$	20	1	8	−2.192	0.028
	40	0	9	−2.666	0.008
	60	1	8	−2.547	0.011
	80	0	9	−2.547	0.008
	100	0	9	−2.666	0.008

only one network ($N+ = 1$) while performing lower on eight networks ($N- = 8$). Although the p-values remain below 0.05 (ranging from 0.015 to 0.038), indicating a statistically significant difference between the two algorithms, the direction of the ranks ($N- > N+$) suggests that CELF generally yields higher influence spread than ABCVNS in these experimental settings. Overall, these results indicate that the performance gaps between ABCVNS and most baseline methods are statistically significant and validate the effectiveness of ABCVNS.

5. Discussion

Comparison with existing work. First, regarding efficiency and accuracy trade-offs, as noted in the Introduction section, traditional greedy algorithms are often constrained by the prohibitive computational overhead of extensive Monte Carlo simulations, despite offering provable approximation guarantees. Conversely, heuristics tend to produce inferior solutions due to their oversight of influence overlap among the candidate seeds. ABCVNS leverages the fitness function to avoid costly simulations, achieving a more favorable balance between efficiency and accuracy. Second, in terms of search strategy, considering that existing meta-heuristics rely on a single and fixed neighborhood search pattern, the proposed algorithm applies the VNS strategy with multiple alternating neighborhood operators effectively to explore the search space. Third, concerning stagnation resolution, the proposed algorithm distinguishes itself by leveraging a fine-grained, individual-centric stagnation control strategy to specifically target and resolve stagnation at the individual level. Complementary to this, a stagnation-aware population management mechanism regenerates less promising individuals from the population-level to sustain search vitality over extended iterations. Finally, empirical validation on nine real-world networks demonstrates that ABCVNS exhibits strong performance comparable to greedy baselines but with orders of magnitude faster convergence across varying topologies.

Theoretical contributions. Methodologically, the proposed algorithm unveils a population-driven intelligent optimization that offers a configurable and extensible template for other combinatorial optimization problems. First, we propose the EGCNC mechanism to overcome the absence of directional guidance in standard ABC perturbation logic. By embedding information from elite and globally optimal individuals into neighborhood perturbations, the EGCNC mechanism concentrates exploration on promising regions. Furthermore, the bifurcated scout strategy retains high-quality nodes from global best solutions to prevent random generation from disrupting the structure of the optimal solution in the current population. Finally, the study theoretically validates the efficacy of tournament selection within the IM, demonstrating that maintaining moderate selection pressure is crucial for preventing the algorithm from becoming trapped in deceptive local optima.

Practical implications. ABCVNS is applicable to decision-making problems in real-world networked systems, where the selection of a limited set of critical nodes is required under resource constraints. In rumor suppression, the problem can be formulated as choosing a subset of users whose control minimizes the propagation of misinformation, and ABCVNS addresses this task by identifying high-impact nodes that serve as prioritized intervention targets, providing a principled basis for allocating resources toward fact-checking dissemination, content regulation, or exposure control. The algorithm is designed to support decision-making rather than operate as a fully autonomous system. It offers a ranked candidate set that can be evaluated based on contextual or domain-specific criteria before intervention, thereby establishing a human-in-the-loop paradigm that facilitates more reliable and context-aware interventions. Similarly, in infrastructure networks, operators face the challenge of mitigating cascading failures under limited monitoring and maintenance capacity. ABCVNS identifies critical components whose failure may induce widespread disruption, providing guidance for prioritizing inspection, reinforcement, or load redistribution to reduce systemic risk while maintaining operational efficiency. In political campaigning, the task involves selecting a subset of individuals to maximize information dissemination under budget and resource constraints. ABCVNS provides a ranked set of influential nodes that enables targeted allocation of outreach efforts toward individuals with higher propagation potential.

Extension to Dynamic Networks. The proposed ABCVNS models the dynamic network as a sequence of network snapshots, which is a common representation where the network evolves over discrete time steps [41]. Such a formulation aligns with practical constraints where computational resources are limited and strategic decisions, such as phased product promotion campaigns or scheduled infrastructure maintenance, are naturally implemented in discrete stages. The effectiveness of the proposed approach relies on the observation that the set of influential nodes tends to remain relatively stable across consecutive snapshots, as observed in [42]. ABCVNS leverages the above property by using high-quality solutions from previous time steps to initialize the search for the current snapshot, rather than restarting from scratch. Consequently, the search is guided toward promising regions of the solution space with reduced computational overhead. The VNS strategy further facilitates localized refinements, allowing the population to adapt to structural changes and progressively converge toward updated optima. By leveraging solution reuse across sequential snapshots, ABCVNS strikes a robust balance between computational tractability and solution adaptability in continuously evolving networks.

Limitations and future work. Although ABCVNS achieves better solution performance compared with other methods, it is important to acknowledge that the introduction of the VNS strategy inevitably incurs

additional computational costs due to iterative local search procedures. Future work could investigate adaptive tuning of neighborhood structures to retain solution quality with reduced runtime, especially for large-scale networks. Moreover, the proposed algorithm currently focuses on single-objective optimization. However, real-world scenarios often involve competing objectives, such as maximizing influence while simultaneously minimizing advertising costs. Future efforts could formulate multi-objective variants of ABCVNS.

6. Conclusion

This paper revitalizes ABC by blending it with the VNS strategy and culminates in a novel intelligent optimization algorithm termed ABCVNS. The algorithm applies the VNS strategy to achieve a dynamic neighborhood switching scheme to strengthen the capacity in local search. To further steer the search toward promising regions, a well-designed elite-global cooperative neighborhood construction mechanism is devised to adjust search directions based on historical information. Extensive experiments on nine networks demonstrate that our algorithm surpasses the state-of-the-art algorithms on various types of networks. These outstanding findings highlight the potential of ABCVNS for IM. Although validated on the IM, its methodological advancements generalize to a variety of combinatorial optimization tasks characterized by curtailed neighborhood diversity or weakened late-stage search performance. The merits underscore the theoretical contribution of advancing the baseline algorithm and offer methodological support for social network analysis and swarm intelligence optimization. Despite these strengths, the algorithm currently faces limitations. The present study is subjected to single-layer networks with a single-objective focus. Future efforts could extend the approach to higher-order networks and formulate multi-objective variants to better capture complex real-world scenarios.

CRediT authorship contribution statement

Jianxin Tang: Writing – review & editing, Investigation. **Fangjie Dong:** Writing – original draft, Software, Methodology. **Chen Wu:** Writing – review & editing, Visualization. **Wenhuan Chen:** Supervision, Resources. **Huiyi Wei:** Formal analysis, Conceptualization.

Availability of material and/or code

Code will be made available on request.

Declaration of Generative AI and AI-assisted technologies in the writing process

Statement: During the preparation of this work, the authors used AI-assisted error correction in order to improve readability and language. After using this service, the authors reviewed and edited the content as needed and took full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (grant numbers 62562044 and 62162040); the Gansu Provincial University Teachers Innovation Foundation (grant number 2024A-024); the Gansu Provincial Science Fund for Distinguished Young Scholars (grant number 23JRRA766); and the Gansu Provincial Science Fund for Technological Innovation Guidance Plan (grant number 24CXGA046).

Data availability

Data will be made available on request.

References

[1] S.S. Singh, D. Srivastva, M. Verma, J. Singh, Influence maximization frameworks, performance, challenges and directions on social network: A theoretical study, J. King Saud Univ.-Comput. Inf. Sci. 34 (2022) 7570–7603.
[2] J.Y. Lee, The effects of social media criticism against public health institutions on trust, emotions, and social media engagement, Proc. Natl. Acad. Sci. 122 (2025) e2422890122.
[3] S.J. Taylor, L. Muchnik, M. Kumar, S. Aral, Identity effects in social media, Nat. Hum. Behav. 7 (2023) 27–37.
[4] M. Jaouadi, L. Ben Romdhane, A survey on influence maximization models, Expert Syst. Appl. 248 (2024) 123429.
[5] D. Kempe, J. Kleinberg, É. Tardos, Maximizing the spread of influence through a social network, in: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2003, pp. 137–146.
[6] J. Ma, J. Hu, Identifying critical nodes in complex networks via a Multi-Scale Influence Spread method, Chaos Solitons Fractals 198 (2025) 116573.
[7] Q. Hu, J. Jiang, H. Xu, M. Kassim, IMNE: Maximizing influence through deep learning-based node embedding in social network, Swarm Evol. Comput. 88 (2024) 101609.
[8] L. Jiao, J. Zhao, C. Wang, X. Liu, F. Liu, L. Li, R. Shang, Y. Li, W. Ma, S. Yang, Nature-inspired intelligent computing: A comprehensive survey, Research 7 (2024) 0442.
[9] Z. Ou, S. Wang, Finding robust and influential nodes on directed networks using a memetic algorithm, Swarm Evol. Comput. 87 (2024) 101542.
[10] I. Khatri, A. Choudhry, A. Rao, A. Tyagi, D.K. Vishwakarma, M. Prasad, Influence Maximization in social networks using discretized Harris' Hawks Optimization algorithm, Appl. Soft Comput. 149 (2023) 111037.
[11] Y. Liu, B. Wang, S. Zhang, P. Tian, H. Zhang, C. Liu, X. Jiang, Influential nodes identification based on Quasi-Laplacian Gravity Model, Chaos Solitons Fractals 200 (2025) 117159.
[12] D. Karaboga, et al., An idea based on honey bee swarm for numerical optimization, J. Math. Sociol. (2005).
[13] P. Hansen, N. Mladenović, Variable neighborhood search: Principles and applications, European J. Oper. Res. 130 (2001) 449–467.
[14] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. Vanbriesen, N. Glance, Cost-effective outbreak detection in networks, in: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2007, pp. 420–429.
[15] W. Chen, Y. Wang, S. Yang, Efficient influence maximization in social networks, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009, pp. 199–208.
[16] C. Borgs, M. Brautbar, J. Chayes, B. Lucier, Maximizing social influence in nearly optimal time, in: Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, 2014, pp. 946–957.
[17] Y. Tang, X. Xiao, Y. Shi, Influence maximization: near-optimal time complexity meets practical efficiency, in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, 2014, pp. 75–86.
[18] Y. Tang, Y. Shi, X. Xiao, Influence Maximization in Near-Linear Time: A Martingale Approach, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 2015, pp. 1539–1554.
[19] Q. Guo, S. Wang, Z. Wei, M. Chen, Influence Maximization Revisited: Efficient Reverse Reachable Set Generation with Bound Tightened, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 2020, pp. 2167–2181.
[20] P. Bonacich, Factoring and weighting approaches to status scores and clique identification, J. Math. Sociol. 2 (1972) 113–120.
[21] L.C. Freeman, Centrality in social networks conceptual clarification, Soc. Networks 1 (1978) 215–239.
[22] D. Chen, H. Su, Identification of Influential Nodes in Complex Networks With Degree and Average Neighbor Degree, IEEE J. Emerg. Sel. Top. Circuits Syst. 13 (2023) 734–742.
[23] X. Zhu, R. Hao, Finding influential nodes in complex networks by integrating nodal intrinsic and extrinsic centrality, Chaos Solitons Fractals 194 (2025) 116278.
[24] B. Zhang, L. lei Meng, C. Lu, Y. yan Han, H. yan Sang, Automatic design of constructive heuristics for a reconfigurable distributed flowshop group scheduling problem, Comput. Oper. Res. 161 (2024) 106432.
[25] C. Fan, Z. Wang, J. Zhang, J. Zhao, X. Meng, Meta-heuristic algorithms for influence maximization: a survey, Evolving Syst. 16 (2025) 24.
[26] Q. Jiang, G. Song, G. Cong, Y. Wang, W. Si, K. Xie, Simulated Annealing Based Influence Maximization in Social Networks, in: Proceedings of the 25th AAAI Conference on Artificial Intelligence, AAAI 2011, 2011, pp. 127–132.

[27] T.K. Biswas, A. Abbasi, R.K. Chakrabortty, An MCDM integrated adaptive simulated annealing approach for influence maximization in social networks, Inform. Sci. 556 (2021) 27–48.
[28] L. Qiu, X. Tian, J. Zhang, C. Gu, S. Sai, LIDDE: A differential evolution algorithm based on local-influence-descending search strategy for influence maximization in social networks, J. Netw. Comput. Appl. 178 (2021) 102973.
[29] E. Zhu, H. Wang, Y. Zhang, K. Zhang, C. Liu, PHEE: Identifying influential nodes in social networks with a phased evaluation-enhanced search, Neurocomputing 572 (2024) 127195.
[30] J. Tang, J. Fu, X. Li, L. Geng, J. Pang, Probing the fitness landscape of the influential nodes for the influence maximization problem in social networks, Swarm Evol. Comput. 97 (2025) 102002.
[31] M. Gong, J. Yan, B. Shen, L. Ma, Q. Cai, Influence maximization in social networks based on discrete particle swarm optimization, Inform. Sci. 367–368 (2016) 600–614.
[32] Q. He, X. Yan, A. Jolfaei, A. Tolba, K. Yu, Y.-K. Fu, Y. Cai, Influence Maximization in Sentiment Propagation With Multisearch Particle Swarm Optimization Algorithm, IEEE Trans. Comput. Soc. Syst. 12 (2025) 1365–1375.
[33] L. Ma, Y. Liu, Maximizing three-hop influence spread in social networks using discrete comprehensive learning artificial bee colony optimizer, Appl. Soft Comput. 83 (2019) 105606.
[34] A. Sheikhahmadi, A. Zareie, Identifying influential spreaders using multi-objective artificial bee colony optimization, Appl. Soft Comput. 94 (2020) 106436.
[35] P. Hansen, N. Mladenovic, A tutorial on variable neighborhood search, Les Cah. Du GERAD ISSN 711 (2003) 2440.
[36] P. Hansen, N. Mladenović, R. Todosijević, S. Hanafi, Variable neighborhood search: basics and variants, EURO J. Comput. Optim. 5 (2017) 423–454.
[37] J. Brimberg, S. Salhi, R. Todosijević, D. Urošević, Variable Neighborhood Search: The power of change and simplicity, Comput. Oper. Res. 155 (2023) 106221.
[38] S. Wang, Y. Jin, MFEA-rcim: A multifactorial evolutionary algorithm for determining robust and influential seeds from competitive networks under structural failures, IEEE Trans. Cybern. 55 (2025) 3624–3636.
[39] S. Wang, J. Tang, X. Tan, M. Li, A multifactorial evolutionary algorithm to detect stably influential seeds from competitive networks under multiple damage scenarios, Swarm Evol. Comput. 99 (2025) 102187.
[40] V.D. Blondel, J.-L. Guillaume, R. Lambiotte, E. Lefebvre, Fast unfolding of communities in large networks, J. Stat. Mech. Theory Exp. 2008 (2008) P10008.
[41] S.H.S. Dizaji, K. Patil, K. Avrachenkov, Influence maximization in dynamic networks using reinforcement learning, SN Comput. Sci. 5 (2024) 169.
[42] X. Liu, X. Liao, S. Li, S. Zheng, B. Lin, J. Zhang, L. Shao, C. Huang, L. Xiao, On the shoulders of giants: incremental influence maximization in evolving social networks, Complexity 2017 (2017) 5049836.